

UNIVERSIDAD DE SANTIAGO DE
COMPOSTELA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA

Estudio de mecanismos cuánticos para la optimización de hiperparámetros en redes neuronales profundas

Autor/a:

Diego Suárez Castro

Tutores:

Manuel Lama Penin

Anselmo Tomás Fernández Pena

Grado en Inteligencia Artificial

Julio 2026

Trabajo de Fin de Grado presentado en la Escuela Técnica Superior de Ingeniería de la Universidad de Santiago de Compostela para la obtención del Grado en Inteligencia Artificial.

Agradecimientos

A mis tutores, Manuel y Tomás, por su inestimable guía, su constante disponibilidad y el rigor metodológico que han aportado a la dirección de este trabajo. Sus valiosas orientaciones críticas y su confianza en mí a lo largo de este proyecto han sido fundamentales para superar los desafíos técnicos y conceptuales que plantea la optimización cuántica y el *deep learning*.

Al Centro de Supercomputación de Galicia (CESGA), con especial mención a Carlos Fernández Sánchez, por su asistencia y por facilitar el acceso a las capacidades de supercomputación del FinisTerra III y a la QPU superconductora de Qmio. La posibilidad de validar experimentalmente este modelo en hardware cuántico real ha sido un pilar indispensable y una oportunidad excepcional en mi formación.

A mi familia, pareja y amigos, por su comprensión incondicional, su apoyo constante y por ser el refugio necesario durante las exigentes jornadas de este trabajo. Gracias por hacer este camino académico mucho más ameno, compartido y enriquecedor.

Resumen

Este trabajo investiga el potencial de la computación cuántica para la optimización de hiperparámetros (HPO) en modelos de aprendizaje profundo, centrándose específicamente en arquitecturas de tipo Transformer aplicadas al monitoreo predictivo de procesos (PPM) con el dataset de *env_permit*. El problema se modela formalmente como una Optimización Binaria Cuadrática (QUBO) de exactamente 30 variables (30 qubits), mapeando un espacio de búsqueda de 2^{30} combinaciones de hiperparámetros a partir de una muestra de 300 configuraciones iniciales (evaluadas clásicamente a lo largo de 50 épocas con validación cruzada, 5 *folds*).

El objetivo principal es explorar la viabilidad de resolvers cuánticos variacionales bajo el ruido físico de la era NISQ, comparando la Optimización Bayesiana clásica frente al algoritmo resolver cuántico variacional (VQE) estándar y la evolución en tiempo imaginario cuántico variacional (VarQITE).

Mediante simulaciones en el supercomputador FinisTerra III y su posterior validación en la unidad de procesamiento cuántico (QPU) real Qmio (32 qubits) del CESGA, se evaluó experimentalmente el comportamiento físico de los algoritmos ante la decoherencia, errores de lectura y ruido térmico. Los resultados revelan una paradoja física y temporal crucial: mientras VQE (sintonizado secuencialmente con COBYLA) diverge en la QPU real debido al impacto de mínimos locales y ruido, VarQITE logra una convergencia robusta y estable. Esta resiliencia se debe al efecto de filtrado exponencial ($e^{-E_n \tau}$) en tiempo imaginario, que atenúa las componentes no deseadas inducidas por el ruido de la QPU superconductora.

Adicionalmente, VarQITE es un orden de magnitud más rápido en hardware real (220s frente a 1500s de VQE) al permitir el envío en paralelo (*batching*) de circuitos, minimizando peticiones a la QPU. Para validar los resultados, se entrenaron en GPU las 55 soluciones óptimas sugeridas (5-Fold CV). El análisis confirma que aunque no haya una dominancia absoluta en la frontera de Pareto, ya que hay hasta 5 soluciones cuánticas (simuladas y reales) siendo solamente 3 clásicas, validando el potencial de la computación cuántica real, que aún está en una fase temprana de desarrollo, pero aparenta ser prometedora tan pronto.

Memoria tipo A – Índice general

1. Introducción	1
1.1. Computación cuántica	2
1.2. Algoritmos cuánticos	4
1.3. Objetivos del trabajo	5
1.4. Implicaciones Transversales del Estudio	6
1.5. Organización de la memoria	6
2. Estado de conocimiento del problema a abordar	9
2.1. Estado del arte de HPO	9
2.1.1. Hiperparámetros relevantes	9
2.1.2. Algoritmos Clásicos de Optimización y Fine-Tuning	10
2.2. Estado del arte en computación cuántica	11
2.2.1. QITE-VQE	13
2.2.2. Posicionamiento del Trabajo	13
3. Materiales	15
3.1. Recursos Hardware	15
3.1.1. Clúster de Supercomputación FinisTerra III (CESGA)	15
3.1.2. Ordenador Cuántico Qmio	16
3.1.3. Hardware de Desarrollo Local	17
3.2. Conjuntos de datos	17
3.2.1. El Registro de Eventos env_permit (CoSeLoG)	17
3.2.2. Oráculo de Rendimiento y Paisaje de Optimización	18
3.2.3. Espacio de Búsqueda y Restricciones Técnicas	19
3.2.4. Paisaje de la Función Objetivo y Formulación Matemática del QUBO	20
3.3. Software y Librerías	22
4. Metodología	25
4.1. Modelización del Paisaje y Codificación del Espacio de Búsqueda	25
4.1.1. Codificación Cuántica y Distribución de Recursos	26
4.2. Pipeline de 5 Fases	27
4.3. Resolvedores Cuánticos Variacionales: VQE y VarQITE	30

4.3.1.	Evolución en Tiempo Imaginario Cuántico Variacional (Var-QITE)	30
4.3.2.	Diseño del Ansatz y Optimización en VQE	31
4.4.	Simulación de Ruido y Decoherencia en la Era NISQ	31
4.4.1.	Simulación mediante Redes de Tensores (MPS)	31
4.4.2.	Canales de Error y Asimetría de Puertas	32
4.4.3.	Niveles de Ruido y Escenarios de Simulación	32
4.4.4.	Mitigación del Ruido mediante Simplificación del Ansatz (reps=1)	33
4.5.	Infraestructura HPC y Paralelismo	33
4.5.1.	Orquestación Distribuida mediante Slurm Job Arrays	34
4.5.2.	Paralelismo Numérico y Afinidad de Hilos	34
4.6.	Evaluación Comparativa y Baselines	35
4.6.1.	Diseño Experimental y Presupuesto de Cómputo	35
5.	Validación y Pruebas	37
5.1.	Diseño de la Campaña Experimental	37
5.2.	Análisis de Tiempos de Ejecución	38
5.3.	Impacto del Ruido en el Tiempo de Simulación	39
5.4.	Comparativa de Precisión Predictiva	40
5.5.	Análisis de la Frontera de Pareto	41
6.	Discusión de los Resultados	43
6.1.	Verificación de las Hipótesis y Cierre de la Brecha Estructural	43
6.2.	Comparativa con los Baselines del Estudio	44
6.2.1.	Frente al Baseline Cuántico: Standard-VQE	44
6.2.2.	Frente al Baseline Clásico: Optimización Bayesiana	45
6.3.	Implicaciones Transversales	46
6.3.1.	Aspectos Éticos y Medioambientales: Hacia una IA Verde	46
6.3.2.	Aspectos Socioeconómicos: Democratización y Competiti- vidad	46
6.3.3.	Aspectos Legales y Culturales: El Nuevo Paradigma del QML	46
7.	Conclusiones y posibles ampliaciones	47
7.1.	Resumen de las Principales Aportaciones	47
7.2.	Suposiciones y Limitaciones del Estudio	48
7.3.	Vías de Mejora y Trabajo Futuro	49
A.	Manuales técnicos	57
A.1.	Estructura de ficheros	57
A.2.	Creación del entorno	61
A.3.	Reproducción de los resultados	61

Índice de figuras

2.1.	Figura del efecto del valor del learning rate sobre la pérdida . . .	10
2.2.	Diferentes ámbitos de aplicación y casos de uso de la computación cuántica con distribución extendida.	12
2.3.	Intersección tecnológica entre la computación cuántica y los paradigmas de <i>Machine Learning</i>	12
4.1.	Esquema metodológico del bucle cerrado clásico-cuántico-clásico empleado para el HPO.	28
5.1.	Análisis comparativo de tiempos medios: optimización del resol- vedor (clásico/cuántico) y entrenamiento real del Transformer en GPU (5-Fold CV).	38
5.2.	Degradación del tiempo de simulación clásica en el FT3 por nivel de ruido, promediando las 5 semillas para cada algoritmo cuántico.	40
5.3.	Comparativa de la precisión media real (accuracy) alcanzada por el EventTransformer bajo los hiperparámetros sugeridos por cada algoritmo.	41
5.4.	Frontera de Pareto HPO: compromiso multiobjetivo entre la pre- cisión de validación y el tiempo medio de entrenamiento (5-Fold CV).	42

Índice de tablas

3.1. Nodos de cómputo principales del clúster FinisTerra III (CESGA).	16
4.1. Parámetros probabilísticos de los modelos de ruido estocástico simulados (e_{1q} : error de 1 qubit, e_{2q} : error de 2 qubits, e_{ro} : error de lectura o readout).	33

Capítulo 1

Introducción

Uno de los grandes problemas de las redes neuronales artificiales es la selección adecuada del valor de ciertos parámetros de la red, llamados hiperparámetros, que mejorarían su rendimiento. Estos parámetros no son aprendidos, por lo que tienen que fijarse previamente al entrenamiento de las redes neuronales artificiales. Los hiperparámetros pueden estar relacionados con la arquitectura o estructura del modelo (número de capas ocultas, función de activación, tamaño de un filtro de convolución...) o con el propio proceso de entrenamiento (tasa de aprendizaje, tamaño de lote...).

Antiguamente, los hiperparámetros se elegían de forma manual bajo el criterio de los expertos que diseñaban la red neuronal. Este sistema tenía la gran desventaja de que requería disponer de personal experto, lo que tardaba gran cantidad de tiempo si la red tiene un tamaño grande y además no asegura encontrar la configuración de hiperparámetros óptima. La tendencia es trabajar con arquitecturas cada vez más grandes y pesadas, es decir, con mayor número de hiperparámetros. Por ello se están usando cada vez más técnicas de aprendizaje automático (ML por sus siglas en inglés), puesto que ahorra el coste de personal a cambio de aumentar el coste computacional.

Un componente clave de ML, y el que será objeto de estudio en este trabajo, es la optimización de hiperparámetros (HPO por sus siglas en inglés). En este componente se abarcan numerosos algoritmos así como estrategias para obtener de manera automática las configuraciones de hiperparámetros óptimas. Por lo que se busca optimizar una función muy costosa de evaluar, ya que conocer su valor exacto para una configuración en concreto de hiperparámetros implica entrenar el modelo N épocas con dicha configuración. Es por esto que HPO es un proceso que, a menudo, tiene un elevado coste computacional.

Debido a que estos hiperparámetros tienen valores continuos, existen infinitos valores que se podrían seleccionar para cada hiperparámetro (por ejemplo, la tasa

de aprendizaje o el número de capas ocultas). Si bien existen ciertas heurísticas que limitan ese número de valores, este sigue siendo muy elevado, de modo que las técnicas de optimización que se pueden desarrollar para elegir los valores más adecuados tienen tiempos de computación muy elevados, lo cual hace que no sea práctica su aplicación.

1.1. Computación cuántica

Los algoritmos cuánticos parten de una base diferente de los algoritmos clásicos, ya que se benefician de los principios de la mecánica cuántica [1]. Para poder entender su funcionamiento en profundidad, conviene entender antes qué es la computación cuántica [2] y qué diferencia tiene de la clásica. La principal diferencia radica en las unidades de medida del ordenador: en computación clásica se opera con bits que se representan de forma binaria con 0 o 1, mientras que en computación cuántica se opera con qubits [3] cuya formulación es:

$$|\psi\rangle = \alpha_0 |0\rangle + \alpha_1 |1\rangle \quad (1.1)$$

También se puede formular el desglose de un qubit, ya que resulta más intuitivo para comprender la ventaja que otorga al transportar una mayor densidad de información:

$$\begin{aligned} |\psi\rangle &= \alpha_0 |0\rangle + \alpha_1 |1\rangle \\ &= \alpha_0 \frac{|+\rangle + |-\rangle}{\sqrt{2}} + \alpha_1 \frac{|+\rangle - |-\rangle}{\sqrt{2}} \\ &= \frac{\alpha_0 + \alpha_1}{\sqrt{2}} |+\rangle + \frac{\alpha_0 - \alpha_1}{\sqrt{2}} |-\rangle \end{aligned} \quad (1.2)$$

Donde α_0 y α_1 son números complejos denominados *amplitudes de probabilidad*. Estos coeficientes deben satisfacer la condición de normalización $|\alpha_0|^2 + |\alpha_1|^2 = 1$, la cual garantiza que la suma de las probabilidades de obtener los estados $|0\rangle$ o $|1\rangle$ al realizar una medición sea la unidad. Esta propiedad, conocida como la *Regla de Born*, es la que permite que un único qubit transporte una densidad de información significativamente superior a la de un bit clásico antes del colapso de su estado [4].

Los qubits son similares en el sentido de representarse también con 0 o 1, pero a diferencia de los bits clásicos, pueden encontrarse en un estado de superposición coherente, cuya fórmula general para un sistema de n qubits es:

$$|\psi\rangle = \alpha_{00\dots 0} |00\dots 0\rangle + \alpha_{00\dots 1} |00\dots 1\rangle + \dots + \alpha_{11\dots 1} |11\dots 1\rangle \quad (1.3)$$

Donde los coeficientes $\alpha_{00\dots 0}, \alpha_{00\dots 1}, \dots, \alpha_{11\dots 1}$ son las amplitudes de probabilidad y pueden tomar valores complejos.

Un ordenador clásico tendría que realizar operaciones secuenciales si quisiera evaluar los estados 0 y 1, mientras que un ordenador cuántico, gracias a la superposición, puede operar sobre ambos estados simultáneamente en una única operación. Como se puede intuir, esto representa una gran ventaja en términos de velocidad de cómputo. Cabe aclarar que un qubit es un sistema cuántico de dos niveles que existe en una superposición de sus estados base ($|0\rangle$ y $|1\rangle$). Al realizar una medición para determinar el estado resultante, la superposición colapsa de forma irreversible en un único estado clásico definido, bien sea 0 o 1.

Otro elemento importante y diferente en la computación cuántica, es el uso de puertas lógicas. Como es bien sabido, los ordenadores clásicos operan con semiconductores CMOS (complementary metal-oxide-semiconductor en inglés) [5]. Estos operadores permiten hacer todo tipo de operaciones, por lo que son muy versátiles, en cambio, en computación cuántica hay operaciones que por lógica matemática son imposibles de hacer, lo cual es una limitación y una desventaja en contra de la computación cuántica. Es por ello que las puertas Pauli-X (equivalente al NOT clásico, para 1 qubit) o también llamadas CNOT (controlled-NOT en inglés, para 2 qubits) [6] son fundamentales y de las más usadas, al hacer una de las operaciones más básicas e importantes de todas, la operación NOT. Esta es su formulación y representación:

$$\text{CNOT: } \begin{array}{c} A \text{---} \bullet \\ | \\ B \text{---} \oplus \end{array} \quad \text{CNOT}_{AB} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad (1.4)$$

Esta puerta lógica es la base de muchas otras ya que por ejemplo, para hacer un SWAP, se puede hacer con 3 puertas CNOT, tal que:

$$\text{SWAP : } \begin{array}{c} A \text{---} \times \\ | \\ B \text{---} \times \end{array} \sim \begin{array}{c} \bullet \quad \oplus \quad \bullet \\ | \quad | \quad | \\ \oplus \quad \bullet \quad \oplus \end{array} \sim \begin{array}{c} \oplus \quad \bullet \quad \oplus \\ | \quad | \quad | \\ \bullet \quad \oplus \quad \bullet \end{array} \quad (1.5)$$

o una puerta c-Z (Pauli-Z) se haría con 2 puertas Hadamard y 1 CNOT, de esta forma:

$$c-Z : \begin{array}{c} A \text{---} \bullet \\ | \\ B \text{---} \bullet \end{array} \sim \begin{array}{c} \boxed{H} \quad \oplus \quad \boxed{H} \\ | \quad | \quad | \\ \bullet \quad \bullet \quad \bullet \end{array} \sim \begin{array}{c} \bullet \\ | \\ \boxed{H} \quad \oplus \quad \boxed{H} \end{array} \quad (1.6)$$

Las puertas CNOT además de para crear los circuitos lógicos y realizar las operaciones, tienen también otra función importante, ya que sirve como unidad de medida en el tamaño de los circuitos. Dado el ruido que conlleva añadir cada puerta, es evidente que se buscarán crear circuitos con el menor número de puertas CNOT posible, es decir de menor tamaño. Cuanto más ruido haya, menor

será la tasa de éxito que se obtenga al ejecutar en un computador cuántico, la fórmula de la Probabilidad de Éxito Estimada (ESP) [7] sería la siguiente:

$$ESP = \prod_{i=1}^{N_{total}} (1 - \epsilon_i) \quad (1.7)$$

En la cual ϵ_i es el error de cada puerta. Se puede representar de esta otra forma la fórmula:

$$P_{exito} \approx (1 - \epsilon_{CNOT})^{N_{CNOT}} \quad (1.8)$$

En donde como se puede ver, a mayor número de puertas CNOT, menor será la tasa de éxito de la ejecución. Partiendo de la anterior fórmula, existe esta que puede llegar a ser más intuitiva aún:

$$P_{exito} \approx e^{-N_{CNOT} \cdot \epsilon_{CNOT}} \quad (1.9)$$

1.2. Algoritmos cuánticos

Una vez comprendido el funcionamiento básico de la computación cuántica, se puede intuir que tipo de ventajas aportarán los algoritmos cuánticos frente a los clásicos. Para comprender mejor las ventajas que pueden aportar, se debe primero entender su funcionamiento, y en qué difieren de los algoritmos clásicos. Como era de esperar, la idea principal de los algoritmos cuánticos es que se aprovechen de los principios cuánticos y gracias a la superposición puedan llegar a soluciones óptimas en menor tiempo que los algoritmos clásicos [8].

Hay que hacer dos grandes distinciones en los algoritmos de optimización investigados, un grupo son los algoritmos cuántico-inspirados [9], [10], los cuales no se pueden ejecutar en ordenadores cuánticos, ni generan circuitos con puertas CNOT. Y por el otro lado, están los algoritmos cuánticos, que si se pueden ejecutar en ordenadores cuánticos. Los algoritmos cuántico-inspirados QIAOs (Quantum-Inspired Optimization Algorithms), son muy útiles ya que presentan ventajas de optimalidad y velocidad frente a los algoritmos clásicos, pese a estar ejecutados ambos en hardware clásico.

Este estudio se centrará sobre todo en el segundo tipo de algoritmos, los cuánticos reales. Aunque cabe destacar la eficacia de los cuántico-inspirados como: QPSO (Quantum-behaved Particle Swarm Optimization, por sus siglas en inglés) [11], [12], QACO (Quantum Ant Colony Optimization, por sus siglas en inglés) [13], [14] o QGA (Quantum Genetic Algorithm, por sus siglas en inglés) [15], [16]. Estos 3 algoritmos funcionan como metaheurísticas ya que obtienen una precisión alta, no óptima, en un tiempo muy bajo a comparación del resto de algoritmos.

En los algoritmos cuánticos se pueden diferenciar varias familias. La más grande de todas es la de algoritmos variacionales VQA donde se encuentran VQE y sus variantes [17], [18], así como QAOA y sus variantes [19]-[23], además de optimizadores sin gradiente como pueden ser FALQON [24], [25], Rotosolve y sus variantes [26], [27], así como FRAXIS [28], [29]. La tercera familia es la de algoritmos de búsqueda cuántica en donde están Grover y sus variantes [30], [31], QA (Quantum Annealing, por sus siglas en inglés) [32], y NV-QWOA (Non-Variational Quantum Walk Optimization Algorithm, por sus siglas en inglés) [33], [34]. La penúltima familia es la de búsqueda de arquitecturas cuánticas donde se encuentran algoritmos como DQAS y sus variantes [35]-[37], así como EQNAS [38] y QuantumNAS [39]. Por último, cabe mencionar a la familia del aprendizaje cuántico (QML), en donde hay algoritmos como Q-GP-UCB (Quantum-Gaussian Process UCB, por sus siglas en inglés), QSVR [40], [41], VQC-RL [42] entre otros varios.

1.3. Objetivos del trabajo

Tras los avances de la computación cuántica junto con los algoritmos cuánticos, se generó una duda, y es si sería posible optimizar los hiperparámetros (HPO) de los modelos usando los paradigmas y beneficios de la computación cuántica. Justamente uno de los objetivos de este trabajo es hacer un estudio de las aproximaciones actuales para la optimización de hiperparámetros en el campo del aprendizaje automático, en general, y en el de las redes neuronales, en particular de Transformers. Además de llegar a seleccionar la combinación óptima del conjunto de hiperparámetros a mejorar, queriendo optimizar tanto el tiempo de entrenamiento (menor tiempo es mejor) como la precisión (mayor tiempo es mejor).

Una vez elegida la estrategia óptima, se requería el objetivo de diseñar e implementar una solución basada en computación cuántica para la optimización de hiperparámetros en redes neuronales artificiales, concretamente, en redes neuronales de propagación hacia adelante (feed-forward). Por lo tanto, otro objetivo es realizar una comparación del rendimiento de la solución basada en computación cuántica con otras soluciones de optimización de hiperparámetros clásicas y cuánticas.

La comparación se hará con el fin de poder estudiar la viabilidad o no de estos algoritmos. Siendo así la finalidad, llegar a implementar y experimentar con algoritmos cuánticos (tanto en simulación como en hardware cuántico real) con el fin de evaluar críticamente la viabilidad y el rendimiento de estas técnicas en la era NISQ [Noisy Intermediate-Scale Quantum], identificando tanto sus ventajas potenciales como sus limitaciones actuales frente a los métodos clásicos.

1.4. Implicaciones Transversales del Estudio

El desarrollo de soluciones híbridas cuántico-clásicas para la optimización de hiperparámetros trasciende el marco estrictamente técnico de la ingeniería de software, entrelazándose con repercusiones éticas, socioeconómicas, legales y culturales que contextualizan la relevancia de esta investigación en la sociedad tecnológica actual.

En el plano **ético y medioambiental**, la optimización de modelos profundos como los Transformers mediante técnicas clásicas tradicionales se enfrenta a un problema de sostenibilidad debido a la elevada huella de carbono derivada del uso ininterrumpido de clústeres de GPUs. La introducción de algoritmos cuánticos variacionales permite abordar este desafío bajo el paradigma de la Inteligencia Artificial Verde (*Green AI*). Al explotar el filtrado por tiempo imaginario, se optimiza la exploración del espacio de estados y se reduce drásticamente el número de entrenamientos iterativos clásicos necesarios, disminuyendo así el consumo energético global del proceso.

Desde una perspectiva **socioeconómica**, esta aproximación favorece la democratización de la Inteligencia Artificial de alta escala. Al reducir los costes computacionales asociados al diseño de redes neuronales, se mitiga el oligopolio de las grandes corporaciones, permitiendo que universidades, centros de investigación públicos y pequeñas o medianas empresas compitan en igualdad de condiciones. Asimismo, la validación de estos modelos en infraestructuras de acceso público, como el supercomputador FinisTerra III y el ordenador cuántico real Qmio del CESGA, subraya el valor estratégico de la inversión pública para garantizar la soberanía tecnológica y fomentar la retención de talento especializado.

Finalmente, en el ámbito **legal y cultural**, la automatización del diseño arquitectónico de software mediante el colapso de estados cuánticos probabilísticos plantea dudas en torno a la propiedad intelectual, la trazabilidad y la regulación de patentes para patrocínios algorítmicos no humanos. Culturalmente, este trabajo impulsa una transición fundamental en la formación técnica, promoviendo el paso desde la programación imperativa clásica hacia el desarrollo de algoritmia cuántica aplicada a problemas reales de MLOps.

1.5. Organización de la memoria

Esta memoria se estructura de la siguiente forma:

- En el **Capítulo 1** se introducen los conocimientos básicos sobre hiperparámetros, computación cuántica y algoritmos cuánticos.

- En el **Capítulo 2** se realiza una revisión del estado del conocimiento sobre optimización de hiperparámetros y la predicción de rendimiento y se analiza en profundidad el algoritmo analizado (QITE-VQE).
- En el **Capítulo 3** se introducen los materiales más relevantes utilizados durante el proyecto como el supercomputador FinisTerra III o el ordenador cuántico Qmio.
- En el **Capítulo 4** se describe la metodología seguida durante el proyecto y se introduce el algoritmo , ideado durante la realización de este trabajo.
- En el **Capítulo 5** se presentan las pruebas y resultados de los algoritmos cuánticos y clásicos junto con un análisis de los mismos, tanto en simulación cuántica como en ejecución real cuántica.
- En el **Capítulo 6** se extraen conclusiones y se proponen líneas de trabajo futuro.

Capítulo 2

Estado de conocimiento del problema a abordar

2.1. Estado del arte de HPO

2.1.1. Hiperparámetros relevantes

La optimización de hiperparámetros no es un problema exclusivo del ámbito del *machine learning* (ML), hay diferentes ámbitos además del aprendizaje automático profundo. Pero tal y como se avanzó en la introducción, esta área está adquiriendo gran relevancia y fuerza, con motivo de la inteligencia artificial. Justamente por ese objetivo de querer entrenar los mejores modelos y con menos errores posibles, los modelos están siendo entrenados con conjuntos de datos masivos (big data), aunque sin un ajuste de hiperparámetros correcto no se estaría exprimiendo el potencial total de un modelo.

Hay gran cantidad de hiperparámetros, por eso es clave la identificación de los más relevantes y de mayor impacto [43] en el entrenamiento de los modelos, ya que sería un coste demasiado elevado tanto temporal como computacional, tratar de ajustar todos los parámetros. Entre los hiperparámetros con mayor impacto está el batch size, que permite ajustar con que cantidad de datos se va a entrenar en el modelo, siendo este siempre múltiplo de 2. Este hiperparámetro además de influir en el aprendizaje del modelo, también es relevante en el tiempo de ejecución, ya que a medida que el valor aumenta, mayor será el tiempo de entrenamiento [44].

Otro hiperparámetro importante es el learning rate, ya que determina el tamaño del paso en el aprendizaje en el descenso de gradiente. Si es un valor demasiado grande no llegará a aprender y sobreajustará, y si es demasiado pequeño, se quedará estancado en óptimos locales. Es por eso fundamental elegir un valor intermedio óptimo para que pueda aprender en el modelo de manera correcta.

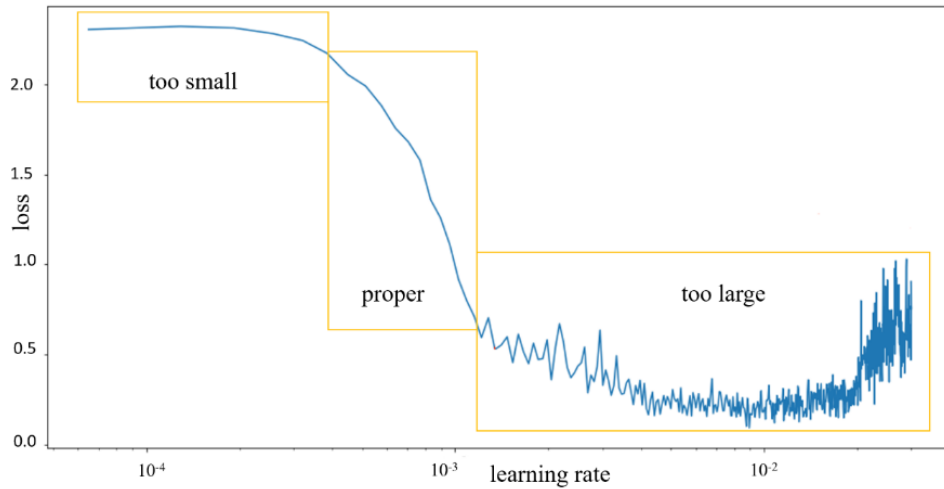


Figura 2.1: Figura del efecto del valor del learning rate sobre la pérdida

En este trabajo, el espacio de búsqueda se extiende a una arquitectura de tipo **Transformer**, incorporando parámetros estructurales de alta complejidad: el **EMB_SIZE** (tamaño del *embedding*), que define la riqueza de la representación interna; el **ATTN_HEADS**, que controla las cabezas de atención paralela; y el número de capas en el codificador y decodificador (**ENC/DEC_LAYERS**), que determinan la capacidad de abstracción de la red.

2.1.2. Algoritmos Clásicos de Optimización y Fine-Tuning

El proceso de *fine-tuning* o ajuste fino es la piedra angular del rendimiento en el aprendizaje profundo moderno. Debido a que un modelo no presenta resultados idénticos ante diferentes dominios de datos, la automatización de la búsqueda de hiperparámetros se vuelve obligatoria. Para resolver este problema, la literatura clásica ha evolucionado desde métodos heurísticos simples hacia estrategias probabilísticas avanzadas.

Búsqueda Exhaustiva y Aleatoria

- **Grid Search:** Realiza una búsqueda exhaustiva en una rejilla predefinida [45]. Aunque garantiza encontrar el mejor punto de la rejilla, padece de la maldición de la dimensionalidad y desperdicia recursos en dimensiones poco sensibles.
- **Random Search:** Propuesto como una alternativa más eficiente, selecciona configuraciones aleatoriamente. Su éxito radica en que permite explorar una mayor variedad de valores en las dimensiones críticas, superando habitualmente al Grid Search [45] con el mismo presupuesto computacional.

Optimización Bayesiana (BO)

La Optimización Bayesiana representa el estado del arte para la optimización de funciones de caja negra costosas de evaluar. A diferencia de los métodos anteriores, BO utiliza la información de evaluaciones pasadas para decidir de manera inteligente cuál será el próximo punto a probar [46]. Este proceso se basa en dos componentes fundamentales:

1. **Modelo Subrogado (Surrogate Model):** Dado que entrenar una red neuronal para evaluar una configuración es extremadamente costoso, BO construye un modelo estadístico que imita el comportamiento de la función real. En este trabajo, cobra especial relevancia el uso de **Random Forest** como modelo subrogado, el cual, mediante el uso de múltiples árboles de decisión, estima no solo el valor esperado de la precisión, sino también la incertidumbre en zonas no exploradas del espacio de búsqueda.
2. **Función de Adquisición:** Es la encargada de guiar la búsqueda. Utiliza la información del modelo subrogado para equilibrar la *exploración* (probar zonas con alta incertidumbre) y la *explotación* (probar zonas donde el modelo predice un buen rendimiento). Algoritmos como *Expected Improvement* (EI) permiten que BO converja hacia el óptimo global con un número de evaluaciones órdenes de magnitud menor que los métodos aleatorios.

Esta sofisticación en los métodos clásicos subrogados proporciona el marco comparativo ideal para este trabajo, donde se evaluará si las propiedades de la computación cuántica pueden ofrecer una ventaja competitiva sobre estos modelos probabilísticos.

2.2. Estado del arte en computación cuántica

La computación cuántica es una herramienta muy novedosa que está demostrando ser muy útil en diversas aplicaciones. Una de las más conocidas es en la farmacología, al ser capaz de computar simulaciones y descubrir nuevos fármacos en un menor tiempo. También en la rama médica al poder ser combinada con inteligencia artificial, y así poder detectar tumores o enfermedades de manera más precisa y veloz. En general, en el ámbito sanitario, químico y biológico, es extremadamente útil la computación cuántica, así como el QML (Quantum Machine Learning, por sus siglas en inglés) [47].

Otro ámbito muy relevante que cambió es la ciberseguridad, ya que los sistemas clásicos de seguridad, no estaban preparados para la potencia de cómputo ni los paradigmas de la computación cuántica. Esta área se ve afectada ya que los sistemas se tienen que modernizar para ser robustos incluso ante ataques cuánticos cibernéticos. Es por eso que se está empezando a hablar de "Quantum-IoT

Security” [48], en donde se menciona que el mundo IoT en el que se vive actualmente necesita actualizarse y reforzar sus sistemas de encriptación y seguridad en general.

Hay usos más variados de la computación cuántica, como pueden ser procesamiento de imágenes, análisis de sentimientos, corrección de errores, *machine learning*... [49]

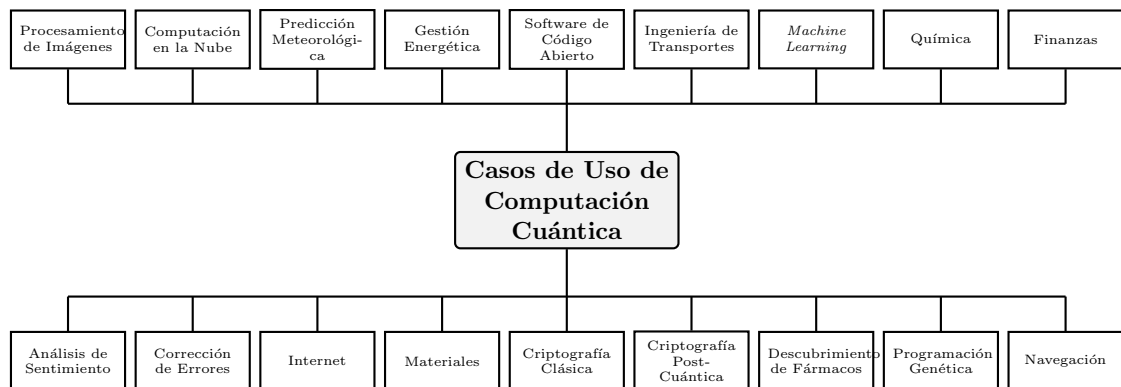


Figura 2.2: Diferentes ámbitos de aplicación y casos de uso de la computación cuántica con distribución extendida.

Este estudio realizado, se va a centrar en el *machine learning* cuántico, es decir, en el QML. El QML es precisamente procesar datos clásicos usando *machine learning* en ordenadores cuánticos. La idea clave es poder aprovechar las ventajas computacionales como: entrelazamiento, superposición, teorema de no clonación e interferencia. Esto ayuda a procesar información de forma más rápida, más allá de los límites clásicos.

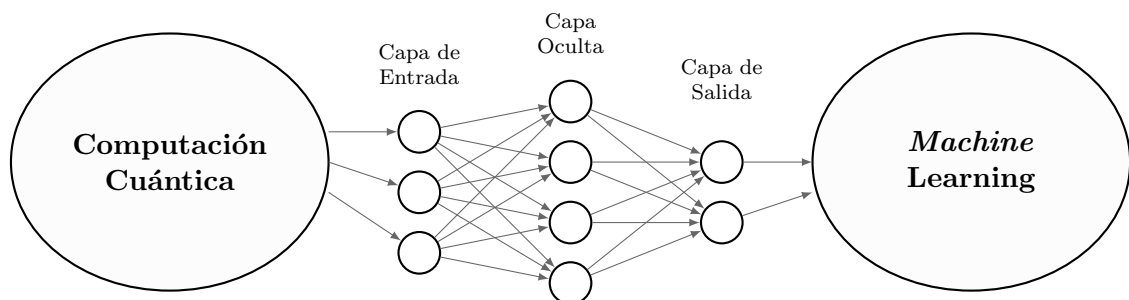


Figura 2.3: Intersección tecnológica entre la computación cuántica y los paradigmas de *Machine Learning*.

2.2.1. QITE-VQE

Existen múltiples implementaciones cuánticas simples, las que transforman un algoritmo clásico en uno cuántico, véase $SVR \rightarrow QSVR$, o también $NN \rightarrow QNN$. Siendo así estas implementaciones más sencillas por poder partir de la base clásica. También hay implementaciones que no tienen su símil clásico, como serían algoritmos tipo QAOA, VQE (cuyos observables se evalúan mediante operadores de Pauli) o Grover, los cuales son algoritmos puramente cuánticos.

Una solución algo más compleja en cuanto a formulación matemática es el algoritmo QITE-VQE (Quantum Imaginary Time Evolution - Variational Quantum Eigensolver, por sus siglas en inglés) .

Este algoritmo usa de base un VQE que aprovecha la evolución en tiempo imaginario para amplificar la probabilidad del estado fundamental antes de calcular la función de coste, reduciendo por lo tanto su profundidad en el circuito que crea [50]. El tiempo imaginario que es lo que le da esa ventaja al baseline de VQE, se puede definir de la siguiente forma:

$$|\psi(\tau)\rangle = \frac{e^{-\tau\hat{H}}|\psi_0\rangle}{\|e^{-\tau\hat{H}}|\psi_0\rangle\|} \quad (2.1)$$

Siendo $|\psi\rangle$ el estado mientras que τ es el tiempo imaginario, además de que $\hat{H}$ es el operador hamiltoniano.

La potencia de esta formulación reside en que el operador $e^{-\tau\hat{H}}$ actúa como un **filtro cuántico**. En el contexto de HPO, el Hamiltoniano $\hat{H}$ codifica el paisaje de precisión de la red neuronal. Este filtro "suprime" las configuraciones de hiperparámetros con bajo rendimiento (estados de alta energía) y amplifica las zonas óptimas.

Desde un punto de vista técnico, esta guía hacia la solución permite alcanzar el estado fundamental con una **menor profundidad de circuito** que un VQE tradicional. En la actual era **NISQ** (*Noisy Intermediate-Scale Quantum*), reducir la profundidad es vital: cada capa adicional y cada puerta **CNOT** introducen ruido térmico y errores de puerta que degradan la fidelidad.

Al minimizar estas métricas, el algoritmo QITE-VQE aumenta las probabilidades de éxito en ejecuciones sobre hardware real como el chip *Qmio*.

2.2.2. Posicionamiento del Trabajo

A diferencia de los estudios que se limitan a comparar precisiones finales en entornos de simulación cuántica puramente idealizada, este trabajo busca llenar

un vacío crítico en la literatura evaluando la **viabilidad estructural y física** de los algoritmos en procesadores reales de la era NISQ. Al reportar el número de puertas CNOT de entrelazamiento y la profundidad del circuito cuántico transpilado, este estudio establece una relación directa entre las especificaciones físicas del hardware cuántico superconductor disponible y el beneficio real obtenido al optimizar modelos de aprendizaje profundo a gran escala (QUBO de 30 variables con 2^{30} combinaciones).

Asimismo, el impacto de la decoherencia cuántica y la atenuación de gradientes se evalúa de manera sistemática inyectando modelos de ruido despolarizante calibrados bajo una estricta proporción física de asimetría 1:10 (desde un escenario ideal hasta tasas de error de despolarización del 1,0 % en CNOTs y un 1,0 % de error de lectura).

Otra diferenciación clave y de gran valor científico en este trabajo es el salto a la práctica real. Lejos de limitar las conclusiones a estimaciones teóricas o simulaciones numéricas clásicas aceleradas en el supercomputador FinisTerra III, las hipótesis de convergencia y factibilidad se validan mediante ejecuciones directas sobre la QPU superconductora real de 32 qubits de Qmio del CESGA. De esta manera, el estudio proporciona un análisis empírico honesto y del máximo rigor académico sobre los límites, la latencia de red de hardware y la tolerancia al ruido real en la frontera híbrida clásico-cuántica.

Capítulo 3

Materiales

En este capítulo se detallan los distintos recursos: hardware, conjuntos de datos y librerías de software, utilizados para el desarrollo del trabajo.

3.1. Recursos Hardware

3.1.1. Clúster de Supercomputación FinisTerra III (CESGA)

La totalidad de los entrenamientos de transformers (Para generar parte del paisaje), los experimentos clásicos y cuánticos simulados de este trabajo han sido ejecutados en el **clúster FinisTerra III**, gestionado por el Centro de Supercomputación de Galicia (CESGA). Este sistema de cómputo de altas prestaciones pertenece a la Red Española de Supercomputación (RES) y está disponible para proyectos de investigación universitaria.

El FinisTerra III cuenta con una arquitectura heterogénea compuesta por distintos tipos de nodos de cómputo, cuyas características se resumen en el Cuadro 3.1.

Tipo de Nodo	#Nodos	CPU	#Cores	Memoria	GPU
Estándar (Ice Lake)	256	2x Xeon 8352Y	64	256 GB DDR4	—
GPU (A100)	66	2x Xeon 8352Y	64	256 GB DDR4	Hasta 8x A100
Visualización (T4)	16	2x Xeon 8352Y	64	256 GB DDR4	1x NVIDIA T4
Fat/SMP (High-Mem)	16	2x Xeon 8352Y	64	2 TB DDR4	—
Optane (SMP-PMem)	1	2x Xeon 8352Y	64	8 TB Optane	—

Tabla 3.1: Nodos de cómputo principales del clúster FinisTerra III (CESGA).

Para la simulación clásica de los circuitos cuánticos se han utilizado de forma intensiva los **nodos de cómputo del FinisTerra III** (con hasta 250 GB de RAM) y, cuando los requisitos computacionales lo exigían, los **nodos Fat o SMP** (Symmetric Multi-Processing) con hasta 2 TB de memoria RAM. Estos recursos son críticos debido al crecimiento exponencial del espacio de estados cuánticos: el vector de estado de un sistema de n cúbits requiere almacenar 2^n amplitudes complejas, lo que para el problema QUBO formulado de exactamente **30 cúbits** supone un consumo base de **16 GB de RAM unúmeros complejos de precisión doble** (128 bits por elemento complejo, complex128) solo para almacenar el estado. En el caso de VarQITE, la necesidad de calcular de manera clásica el Tensor Geométrico Cuántico (QGT) y los gradientes variacionales eleva drásticamente el uso de memoria en simulación clásica, requiriendo nodos de gran memoria.

La planificación y gestión de los trabajos se realiza mediante el gestor de colas **SLURM** (Simple Linux Utility for Resource Management). Las simulaciones de barrido de ruido clásica se enviaron a la partición **medium** (con un límite de 16 horas por trabajo), mientras que las ejecuciones del Transformer neural y la Optimización Bayesiana se ejecutaron en la partición **short** en GPUs Nvidia A100.

3.1.2. Ordenador Cuántico Qmio

Para evaluar la viabilidad en hardware real y trascender la simulación numérica, se ha dispuesto de acceso a **Qmio**, el ordenador cuántico de tecnología superconductora del CESGA integrado en el entorno híbrido de supercomputación desde 2024.

Este sistema híbrido (HPCQC) se encuentra directamente conectado con el supercomputador FinisTerra III mediante una red de interconexión de latencia ultra-baja, permitiendo orquestar el bucle clásico-cuántico de optimización

variacional.

Las características técnicas y operativas de Qmio que fundamentan el diseño experimental de este trabajo son las siguientes:

- **Unidad de Procesamiento Cuántico (QPU):** Qmio integra un chip cuántico de 32 cúbits superconductores de tipo *transmon*. Para nuestro experimento de HPO de 30 variables binarias, el algoritmo se ejecutó sobre **30 cúbits activos**, lo que permitió dejar un margen de seguridad de 2 cúbits libres. Esto facilitó que el transpilador de Qiskit seleccionase de manera dinámica y automática el subconjunto de cúbits físicos con menores tasas de error de puerta y mejores tiempos de decoherencia del chip en el momento de la ejecución.
- **Operaciones y Medidas Nativas (Hamiltoniano de Ising):** El acoplamiento cuadrático de la matriz QUBO se mapea directamente al procesador mediante operadores de espín de Pauli Z y ZZ . Para estimar el valor esperado de la energía en cada paso variacional, se utilizó la primitiva **Estimator** de Qiskit, la cual se comunica con el sistema de control de Qmio para lanzar y medir de forma nativa los circuitos cuánticos parametrizados.
- **Entorno de Ejecución e Integración de Software:** En lugar de emuladores externos, la validación del código y la simulación clásica con y sin ruido térmico y de despolarización se ejecutaron en el supercomputador FinisTerra III usando la librería **Qiskit Aer**. Para la ejecución en hardware real de Qmio, se cargaron los módulos de entorno del CESGA (**qmio-run** y **qmio-tools**) que exponen los backends físicos integrados en el middleware cuántico del criostato.

3.1.3. Hardware de Desarrollo Local

Para el desarrollo, pruebas y análisis de resultados se ha utilizado un ordenador personal con sistema operativo Windows 11, con acceso a los ficheros del clúster mediante protocolos SSH y SFTP para la transferencia de código y datos.

3.2. Conjuntos de datos

3.2.1. El Registro de Eventos **env_permit** (CoSeLoG)

Para el desarrollo experimental de este trabajo, el conjunto de datos inicial utilizado es el registro de eventos real denominado **env_permit** (*Environmental Permit*). Este dataset es un estándar de referencia internacional en el ámbito de la minería de procesos (*Process Mining*) y el monitoreo predictivo de procesos (PPM). Su origen se enmarca dentro del proyecto de investigación holandés

CoSeLoG [51], el cual recopiló las trazas de ejecución correspondientes a los procesos de solicitud y tramitación de **licencias ambientales y de edificación** en diversos municipios de los Países Bajos.

En la estructura interna de la carpeta `data/event_log/splitted/`, el dataset se encuentra segmentado en 15 archivos comprimidos en formato estándar `.xes.gz` (arquitectura XML de intercambio de logs de eventos, IEEE 1849-2016), correspondientes a las particiones de entrenamiento, validación y prueba para cada uno de los 5 folds del estudio experimental (`train_fold[0-4]_env_permit.xes.gz`, `val_fold[0-4]_env_permit.xes.gz` y `test_fold[0-4]_env_permit.xes.gz`).

El análisis de estos ficheros revela que cada evento dentro de una traza (*case*) cuenta con la siguiente estructura multidimensional de atributos:

- **concept:name (Actividad):** Variable categórica que identifica la acción específica ejecutada en el flujo administrativo (por ejemplo, recepción de la solicitud, comprobación de requisitos legales, subsanación de errores por el solicitante, dictamen técnico e informe de resolución).
- **org:group (Grupo Organizativo):** Atributo que clasifica el departamento municipal o grupo de trabajo que interviene en la acción (como *Back Office*, *Front Office* o el equipo de inspección de campo).
- **org:resource (Recurso):** Identificador categórico que registra de manera unívoca al funcionario o inspector concreto responsable de la firma y ejecución del evento.
- **time:timestamp (Marca Temporal):** Registro temporal con resolución de milisegundos a partir del cual el pipeline de datos calcula las variables continuas auxiliares de duración: la diferencia de tiempo desde el inicio del caso (`time_from_start`) y la diferencia temporal respecto al evento inmediatamente anterior en la secuencia (`time_from_prev`).

El pipeline del **EventTransformer** procesa estos atributos de forma integrada. Los datos categóricos (*concept:name*, *org:group* y *org:resource*) se proyectan a espacios densos de embeddings individuales, mientras que las marcas de tiempo normalizadas se incorporan como variables numéricas continuas. Esta representación multivariante permite al Transformer modelar simultáneamente las dependencias del flujo de trabajo, los patrones de comportamiento de los recursos humanos y los retardos temporales del proceso.

3.2.2. Oráculo de Rendimiento y Paisaje de Optimización

Para evaluar la eficacia de los algoritmos de optimización de manera rigurosa, este trabajo emplea un enfoque de **Paisaje de Optimización Parcial**. En lugar

de realizar un entrenamiento completo de la red neuronal en cada paso de evaluación de los resolvidores cuánticos (lo cual sería computacionalmente inviable al requerir a veces horas de cómputo por prueba), se utiliza un modelo matemático que aproxima de forma continua el coste del modelo.

El núcleo de este oráculo de rendimiento es el dataset maestro `resultados_hpo.csv`, el cual contiene los resultados reales del entrenamiento y evaluación del EventTransformer sobre el registro `env_permit`. Este conjunto consta de 301 combinaciones discretas de hiperparámetros evaluadas exhaustivamente mediante **validación cruzada de 5 folds durante 50 épocas por fold** en las GPUs Nvidia A100 del supercomputador FinisTerra III, registrando el accuracy final (basado en la similitud de Damerau-Levenshtein), la pérdida de entropía cruzada y el tiempo físico de ejecución.

A partir de este conjunto de datos inicial, se construye parte del paisaje continuo mediante un ajuste estadístico por regresión multivariable con penalización **Lasso L1 con validación cruzada** (implementado en `generate_qubo.py`). Este modelo estima los coeficientes de acoplamiento cuadráticos y lineales para definir una superficie matemática de costo. La superficie se traduce directamente en una formulación de optimización binaria cuadrática sin restricciones (**QUBO**) de exactamente **30 variables binarias (30 cúbits)**, empleando una codificación One-Hot para parametrizar de forma simétrica las 5 opciones discretas correspondientes a cada uno de los 6 hiperparámetros del Transformer. De esta manera, la minimización de la energía del Hamiltoniano cuántico equivale directamente a encontrar la combinación óptima de hiperparámetros del Transformer neuronal que minimice el tiempo físico de entrenamiento al mismo tiempo que maximice la precisión (una frontera de Pareto en resumen).

3.2.3. Espacio de Búsqueda y Restricciones Técnicas

El espacio de búsqueda discreto se define en el script de muestreo `sampler_hpo.py` para sintonizar los **6 hiperparámetros** críticos de la arquitectura EventTransformer:

1. `BATCH_SIZE`: Tamaño del lote de entrenamiento.
2. `LEARNING_RATE`: Tasa de aprendizaje del optimizador.
3. `EMB_SIZE`: Dimensión del modelo o tamaño de los embeddings.
4. `ATTN_HEADS`: Número de cabezas de atención.
5. `ENC_LAYERS`: Número de capas del codificador.
6. `DEC_LAYERS`: Número de capas del decodificador.

Una restricción fundamental para la viabilidad computacional de la arquitectura Transformer es el **requisito de consistencia estructural**: la dimensión del embedding (`EMB_SIZE`) debe ser divisible de manera exacta por el número de cabezas de atención (`ATTN_HEADS`). Si esta condición no se cumple (`EMB_SIZE % ATTN_HEADS != 0`), el módulo de autoatención multicanal no puede dividir simétricamente los subespacios de representación, provocando un fallo crítico en el entrenamiento clásico con PyTorch.

En la realidad experimental de este trabajo, esta restricción técnica se resolvió en la fase previa al entrenamiento:

- **Filtrado a priori:** El script `sampler_hpo.py` calcula el producto cartesiano de todas las combinaciones posibles del espacio y descarta de forma automática aquellas configuraciones no válidas.
- **Generación de la Muestra:** A partir de las combinaciones que cumplen la condición de divisibilidad, se realiza un muestreo determinista uniforme de 300 configuraciones válidas, exportadas a `pool_jobs.csv` y `pool_jobs.json` como índices para el Slurm Job Array.
- **Restricciones en el QUBO:** Por otra parte, la restricción de que los resolvers clásicos o cuánticos elijan exactamente una (y solo una) opción de entre las 5 disponibles para cada uno de los 6 hiperparámetros (restricción One-Hot) se impone directamente en la matriz cuadrática mediante la adición del término de penalización de Lagrange (λ) en el Hamiltoniano cuántico final generado por `generate_qubo.py`.

Aquí tienes la versión unificada de la subsección, integrando la formulación clásica del QUBO, la transformación de coordenadas a variables de espín, el Hamiltoniano de Ising y la medición del valor esperado mediante ‘Estimator’.

Toda la estructura matemática está correctamente formateada con entornos ‘equation’ numerados y sus correspondientes etiquetas ‘ para el índice:

3.2.4. Paisaje de la Función Objetivo y Formulación Matemática del QUBO

Para la optimización de los hiperparámetros, se define una función de coste multiobjetivo que busca un equilibrio de Pareto óptimo entre la calidad predictiva de la red neuronal y su coste computacional (MLOps). Específicamente, el objetivo es **maximizar la precisión** del EventTransformer en la predicción de la siguiente actividad del proceso y **minimizar el tiempo físico de entrenamiento** de los 5 folds en paralelo sobre el supercomputador FinisTerra III.

Para formular esto como un problema de minimización clásica, la función de coste multiobjetivo $f^{(m)}$ para cada muestra experimental m se define como:

$$f^{(m)} = -(1 - \beta) \cdot \text{Acc}^{(m)} + \beta \cdot \text{Time}_{\text{norm}}^{(m)} \quad (3.1)$$

Donde:

- **Acc^(m)**: Es la precisión de validación final (*Accuracy*) promedio obtenida por el Transformer, calculada mediante similitud de secuencias de Damerau-Levenshtein. El signo negativo fuerza al optimizador a maximizar esta métrica.
- **Time_{norm}^(m)**: Es el tiempo de ejecución físico en segundos normalizado en el rango $[0, 1]$ para el entrenamiento de los 5 pliegues a lo largo de las 50 épocas por pliegue. Su signo positivo incentiva la minimización del coste computacional.
- **β** : Es el factor de ponderación multiobjetivo, configurado en $\beta = 0,3$ para dar un peso prioritario del 70 % a la precisión predictiva del modelo y un 30 % a la velocidad y eficiencia de entrenamiento.

A partir del *pool* de datos históricos, la regresión regularizada Lasso L1 reconstruye esta función de coste sobre un espacio continuo aproximado mediante un polinomio de segundo orden. Esto permite formalizar el paisaje de optimización como un problema de Optimización Binaria Cuadrática sin Restricciones (**QUBO**) sobre un vector de variables binarias $\mathbf{x} = (x_1, \dots, x_{30})^T \in \{0, 1\}^{30}$ con codificación One-Hot:

$$E(\mathbf{x}) = \sum_{i=1}^{30} Q_{ii}x_i + \sum_{i=1}^{30} \sum_{j=i+1}^{30} Q_{ij}x_ix_j \quad (3.2)$$

Donde los elementos de la matriz Q de tamaño 30×30 (`matriz.qubo.csv`) incorporan los términos lineales y cuadráticos ajustados por Lasso, junto con el multiplicador de Lagrange ($\lambda = 0,659$) que penaliza fuertemente en la diagonal ($Q_{ii} = h_i - \lambda$) y fuera de ella ($Q_{ij} = J_{ij} + 2\lambda$ para variables del mismo grupo) cualquier violación a las restricciones de exclusividad One-Hot.

Para su procesamiento en la QPU de Qmio, las variables binarias clásicas se proyectan al espacio cuántico de Pauli mediante la transformación:

$$x_i \rightarrow \frac{I - Z_i}{2} \quad (3.3)$$

Sustituyendo esta equivalencia en la Ecuación 3.2, se obtiene el Hamiltoniano cuántico de Ising observable $\hat{H}$ en términos de Pauli Z_i y acoplamientos $Z_i Z_j$:

$$\hat{H} = \sum_{i=1}^{30} h_i Z_i + \sum_{i=1}^{30} \sum_{j=i+1}^{30} J_{ij} Z_i Z_j \quad (3.4)$$

El valor esperado de la energía de este Hamiltoniano para un estado cuántico variacional $|\psi(\theta)\rangle$ se evalúa mediante la primitiva **Estimator**:

$$E(\theta) = \langle \psi(\theta) | \hat{H} | \psi(\theta) \rangle \quad (3.5)$$

Este modelado evita por completo la necesidad de almacenar en disco o RAM clásicos el vector de costo exponencial de 2^{30} configuraciones, delegando la estimación y medida de la energía directamente en las correlaciones de espín físicas de la QPU superconductora de Qmio.

3.3. Software y Librerías

El lenguaje de programación empleado en el desarrollo del trabajo es **Python**, estructurado en un entorno híbrido para asegurar la compatibilidad con las diferentes plataformas físicas: se validó y ejecutó el pipeline de aprendizaje profundo y entrenamiento en GPU con **Python 3.10.8** por estabilidad y rendimiento, mientras que los resolvedores cuánticos y la comunicación directa con el software de control de la QPU real de Qmio se certificaron y ejecutaron sobre **Python 3.9.9** debido a requisitos estrictos de la biblioteca **qmiotools** de la QPU del CESGA. A continuación se describen las bibliotecas más relevantes integradas en el pipeline híbrido clásico-cuántico del proyecto.

PyTorch y CUDA

Núcleo del componente clásico de aprendizaje profundo:

- **Archivos de uso:** `src/embeddings.py`, `src/encoders_and_decoders.py`, `src/output_layers.py`, `src/training.py`, `src/evaluation.py`, `src/utils.py`, `src/train_hpo.py` y `src/config.py`.
- **Propósito:** Construcción de la arquitectura neuronal profunda **EventTransformer** (capas de autoatención con RoPE) y bucles de entrenamiento y validación cruzada. Conexión acelerada mediante **torch.cuda** en las GPUs NVIDIA A100 del FinisTerae III. El archivo `config.py` la utiliza para establecer semillas globales reproducibles en la GPU y CPU.

PM4Py (*Process Mining for Python*)

Biblioteca estándar de minería de procesos:

- **Archivos de uso:** `src/data_processors/event_logs.py` (e indirectamente invocado por `src/train_hpo.py`).
- **Propósito:** Lectura nativa de archivos XML del log real mediante `pm4py.read_xes`, conversión de estructuras con `pm4py.convert_to_event_log` y parseo de trazas del proceso predictivo.

Qiskit y extensiones (Ecosistema cuántico de IBM)

Estructuración del Hamiltoniano, circuitos variacionales y resolvedores cuánticos:

- **Archivos de uso:** `src/solve_vqe.py` y `src/solve_qite.py`.
- **Qiskit Optimization:** Modelado del problema con `QuadraticProgram` y traducción cuántica/decodificación clásica mediante `MinimumEigenOptimizer`.
- **Qiskit Algorithms:** Instanciación de resolvedores cuánticos `SamplingVQE` y `VarQITE` (con el principio de `McLachlan ImaginaryMcLachlanPrinciple`) empleando el ansatz parametrizado `RealAmplitudes`.
- **Qiskit Gradients:** Uso de `LinCombQGT` y `LinCombEstimatorGradient` en `VarQITE` para estimar gradientes cuánticos por combinación lineal, previniendo caídas por falta de memoria (OOM).
- **Qiskit Primitives:** Definición de las interfaces de muestreo y estimación `Sampler` y `Estimator` para simulación o conexión directa a hardware cuántico real.

Qiskit Aer Motor clásico de simulación y emulación cuántica de alto rendimiento:

- **Archivos de uso:** `src/solve_vqe.py` y `src/solve_qite.py`.
- **Propósito:** Simulación cuántica acelerada usando redes de tensores mediante el simulador de estados `matrix_product_state`. Configuración del modelo de ruido físico depolarizante y errores de lectura con la clase `NoiseModel`.

QmioTools (Middleware del CESGA) Control físico e integración cuántica del criostato:

- **Archivos de uso:** `src/solve_vqe.py` y `src/solve_qite.py`.
- **Propósito:** Facilitar la conectividad directa de red con la QPU de Qmio mediante `QmioBackend` (enviando pulsos físicos de microondas) y el backend simulado de pruebas `FakeQmio`.

Scikit-Optimize (skopt) Optimización bayesiana clásica secuencial:

- **Archivos de uso:** `src/solve_bayes_opt.py`.
- **Propósito:** Ejecución del baseline clásico de comparación cuántica importando la función `gp_minimize` e `Integer` para localizar el mínimo del espacio clásico QUBO.

Scikit-Learn (sklearn) Aprendizaje automático e inferencia estadística:

- **Archivos de uso:** `src/generate_qubo.py` y `src/data_processors/attributes.py`.
- **Propósito:** Ajustar la regresión lineal cuadrática regularizada de segundo orden mediante `LassoCV` con validación cruzada para extraer los coeficientes del QUBO, y preprocesar/normalizar las variables continuas del log con `StandardScaler`.

SciPy Optimización clásica libre de gradiente:

- **Archivos de uso:** `src/solve_vqe.py` (integrado a través de los optimizadores de Qiskit).
- **Propósito:** Bucle variacional clásico de optimización de los ángulos del ansatz cuántico empleando la rutina local de aproximaciones lineales `COBYLA` acotada a 100 iteraciones.

Gestión y visualización de datos (Pandas, NumPy y Matplotlib)

Herramientas de soporte analítico y representación:

- **Archivos de uso:** Todos los solucionadores (`solve_*.py`), `src/generate_plots.py`, `src/data_processors/event_logs.py`, `src/data_processors/attributes.py`, `src/evaluation.py` y `src/config.py`.
- **Pandas:** Tratamiento de datasets de logs de eventos, almacenamiento intermedio de datos, y lectura y consolidación en archivos CSV.
- **NumPy:** Soporte general para arrays de datos multidimensionales, transformaciones matriciales de qubits, inicialización de semillas aleatorias de NumPy en `config.py` y cálculos algebraicos auxiliares.
- **Matplotlib:** Renderizado directo de todos los gráficos científicos de alta resolución (frontera de Pareto, curvas de degradación por ruido y trayectorias energéticas).

fcntl y argparse (Módulos de Sistema Unix/Python)

Estructuras de control de bajo nivel y ejecución:

- **Archivos de uso:** `src/train_hpo.py`.
- **Propósito:** El módulo `fcntl` gestiona bloqueos de archivo exclusivos (*exclusive file locking* con `flock`) para evitar que los Job Arrays de Slurm corrompan los datos al escribir concurrentemente en `resultados_hpo.csv`. El módulo `argparse` maneja los parámetros clásicos pasados por terminal.

Capítulo 4

Metodología

En este capítulo se detalla el marco metodológico diseñado para la optimización de hiperparámetros (HPO) en modelos de aprendizaje profundo mediante computación cuántica variacional. La investigación se estructura en un flujo de trabajo integral cerrado clásico-cuántico-clásico, que abarca desde la modelización del paisaje continuo subrogado de rendimiento hasta la validación y ejecución experimental en la unidad de procesamiento cuántico (QPU) superconductora real Qmio. El objetivo principal es proporcionar un entorno de experimentación riguroso que permita evaluar la viabilidad y factibilidad de los resolvedores cuánticos variacionales VQE y VarQITE en un espacio de búsqueda de exactamente **30 qubits** bajo condiciones de ruido cuántico e incoherencia de la era NISQ, y comparar sus resultados frente a la Optimización Bayesiana clásica de referencia.

4.1. Modelización del Paisaje y Codificación del Espacio de Búsqueda

Uno de los pilares centrales de la metodología es el diseño de un modelo de alta fidelidad para representar el rendimiento de arquitecturas Transformer. Se ha adoptado esta aproximación por considerarla absolutamente indispensable, dado que el coste computacional de entrenar y evaluar el EventTransformer mediante validación cruzada clásica en cada paso de optimización del resolvedor cuántico (miles de consultas iterativas) resultaría imposible.

Para construir este paisaje, se ha partido del conjunto de datos histórico `resultados_hpo.csv`. Este dataset reúne los resultados de 301 combinaciones discretas reales del EventTransformer entrenado sobre el registro `env_permit` mediante validación cruzada de 5 folds a lo largo de 50 épocas por fold en el supercomputador FinisTerra III. Con el fin de estructurar este paisaje sobre una función cuadrática continua sin restricciones (**QUBO**), se entrena una regresión regularizada de segundo orden utilizando la clase `LassoCV` (implementada

en `generate_qubo.py`). El modelo estima con alta fidelidad y generalización los coeficientes del Hamiltoniano cuántico (los campos lineales locales h_i y los acoplamientos cuadráticos J_{ij} de la matriz Q de tamaño 30×30), logrando inducir dispersión (*sparsity*) gracias a la penalización L1 para esparcir la matriz final.

Dada la naturaleza del espacio de búsqueda, que abarca 2^{30} combinaciones posibles, el diseño cuántico evita por completo los cuellos de botella de memoria de la fuerza bruta clásica (la cual requeriría almacenar y memory-mapear archivos binarios gigantescos de gigabytes de tamaño en disco). En su lugar, el paisaje parcial se codifica algebraicamente como un observable Hamiltoniano cuántico de espín $\hat{H}$ empleando combinaciones lineales de operadores de Pauli (clase `SparsePauliOp` de Qiskit). La primitiva `Estimator` evalúa de manera compacta el valor esperado de la energía $\langle \hat{H} \rangle$ realizando estimaciones locales de las correlaciones físicas de los cúbits, eliminando de raíz la necesidad de instanciar o tabular los estados no factibles en la RAM de los nodos clásicos de computación.

4.1.1. Codificación Cuántica y Distribución de Recursos

Para formalizar la traducción del espacio clásico de HPO al dominio cuántico, en lugar de una codificación binaria directa basada en potencias de dos (la cual induciría fuertes asimetrías y discontinuidades en el paisaje de optimización) se ha implementado un esquema de **codificación One-Hot simétrica**.

El espacio de búsqueda discreto se compone de $P = 6$ hiperparámetros, asignando a cada uno exactamente $K_p = 5$ opciones discretas de configuración. Cada opción se asocia de forma unívoca a una variable binaria $x_i \in \{0, 1\}$, lo que resulta en un sistema de exactamente:

$$N = \sum_{p=1}^6 K_p = 30 \text{ variables binarias (30 cúbits)} \quad (4.1)$$

El vector binario global $\mathbf{x} = (x_1, \dots, x_{30})^T$ se divide en 6 grupos disjuntos S_p (para $p = 1, \dots, 6$). Para reconstruir el vector clásico de hiperparámetros a partir de una configuración binaria de espines en la QPU, se aplica la transformación lógica:

$$h_p = \text{Space}_p \left[\arg \max_{j \in \{0, \dots, 4\}} (x_{5(p-1)+j+1}) \right] \quad (4.2)$$

Donde Space_p representa el conjunto ordenado de 5 opciones de configuración correspondientes al hiperparámetro p .

Para garantizar que los resolvers variacionales cuánticos converjan únicamente hacia configuraciones físicamente interpretables y viables (activando exactamente una sola opción por hiperparámetro), se incorpora un término de penalización cuadrático de tipo **Lagrange** en la formulación de coste. La restricción

de exclusividad One-Hot para cada grupo S_p exige que:

$$\sum_{i \in S_p} x_i = 1, \quad \forall p \in \{1, \dots, 6\} \quad (4.3)$$

Para integrar esta restricción sin restricciones explícitas en el Hamiltoniano de observables cuánticos, se añade una penalización cuadrática con un multiplicador de Lagrange $\lambda > 0$:

$$H_{\text{penalty}}(\mathbf{x}) = \lambda \sum_{p=1}^6 \left(\sum_{i \in S_p} x_i - 1 \right)^2 \quad (4.4)$$

El valor de $\lambda = 0,659$ actúa como una barrera energética que penaliza cuadráticamente cualquier estado cuántico de prueba $|\psi\rangle$ cuya medida proyectiva active múltiples valores de un mismo hiperparámetro (violaciones One-Hot) o no active ninguno, forzando a los algoritmos cuánticos a colapsar en el espacio de configuraciones factibles.

4.2. Pipeline de 5 Fases

El diseño experimental de este trabajo se articula como un **bucle cerrado clásico-cuántico-clásico** implementado a lo largo de 5 fases metodológicas secuenciales. La arquitectura completa del flujo de trabajo se ilustra en el siguiente diagrama vectorial:

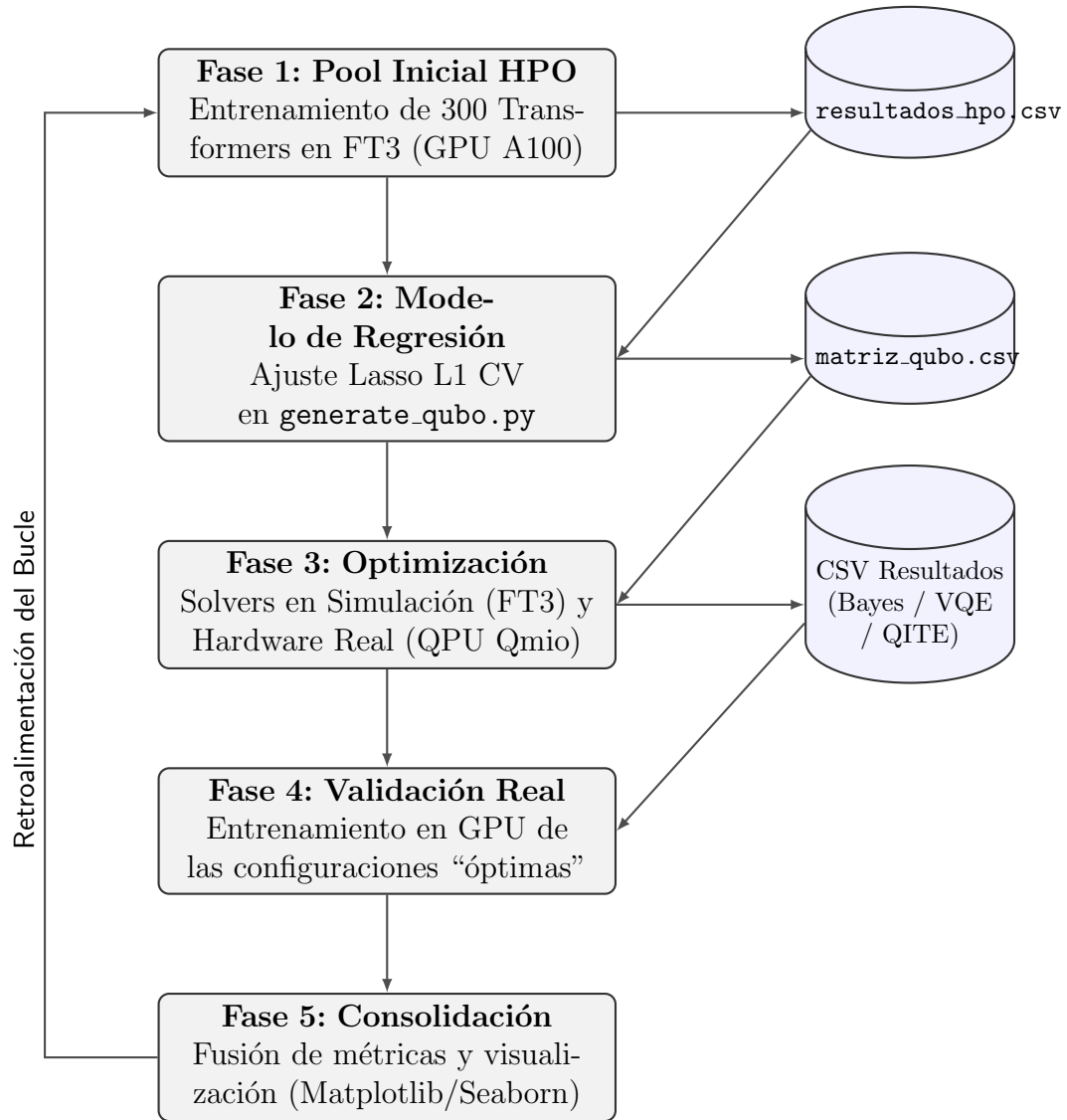


Figura 4.1: Esquema metodológico del bucle cerrado clásico-cuántico-clásico empleado para el HPO.

A continuación, se describen detalladamente las actividades ejecutadas en cada una de las 5 fases:

1. **Fase 1: Datos y Pool Inicial HPO:** Se procesa el registro de eventos en formato XES `env_permit.xes.gz` y se extraen las trazas. Se genera una base de datos histórica mediante el entrenamiento exhaustivo de 300 combinaciones aleatorias válidas del EventTransformer bajo validación cruzada de 5 folds (50 épocas por fold) en el supercomputador FinisTerra III, acumulando un coste computacional de aproximadamente 25 ho-

ras de GPU e identificando el accuracy y tiempos para registrar el dataset `resultados_hpo.csv`.

2. **Fase 2: Modelado del Paisaje y Construcción del QUBO:** El dataset de rendimiento es cargado en `generate_qubo.py` para entrenar un regresor Lasso L1 con validación cruzada. El ajuste estadístico produce los coeficientes que mapean la superficie continua al modelo matemático **QUBO de 30 qubits**. Se ensambla la matriz final de acoplamientos y campos locales Q agregando de forma exacta el multiplicador de Lagrange ($\lambda = 0,659$) para castigar energéticamente las violaciones One-Hot.
3. **Fase 3: Optimización mediante Solvers:** Con la matriz Q de 30 qubits definida, se ejecutan de manera independiente los algoritmos cuánticos variacionales (VQE y VarQITE) y el resolvidor clásico de Optimización Bayesiana:
 - Se realizan barridos de simulación clásicos en el FT3 bajo 4 niveles de ruido.
 - Se ejecuta la optimización física real en la QPU superconductora de Qmio lanzando las tareas por semillas independientes (**SINGLE.SEED**) para maximizar el rendimiento, lanzando de manera independiente cada una de las 5 semillas (1001, 123, 99, 42 y 7).

Las configuraciones óptimas sugeridas por cada resolvidor y semilla se almacenan en sus respectivos archivos de resultados en formato .CSV.

4. **Fase 4: Bucle de Validación Real en GPU:** Para cerrar el bucle clásico-cuántico y verificar si los hiperparámetros óptimos de simulación en el FT3 y en el Qmio se traducen en mejores redes neuronales, se recuperan las configuraciones sugeridas por los resolvidores así como de optimización bayesiana y se ejecutan entrenamientos reales del EventTransformer en la GPU del FT3 (5 folds, 50 épocas). Esto permite evaluar la precisión real del modelo frente a la precisión estimada, así como ver el tiempo real de entrenamiento del transformer.
5. **Fase 5: Consolidación y Visualización:** Las métricas reales de validación (exactitud, pérdida, tiempo físico de GPU) se inyectan en los archivos CSV de resultados, reemplazando los valores de control temporales (se usaba -1 de manera temporal en donde no había precisión y tiempo reales entrenados). Los resultados unificados se comparan y analizan visualmente mediante la generación de figuras y curvas de rendimiento ejecutando el script `generate_plots.py` (basado en Matplotlib y Seaborn).

4.3. Resolvedores Cuánticos Variacionales: VQE y VarQITE

El núcleo cuántico de la investigación radica en la evaluación y comparación de dos de los resolvedores cuánticos variacionales más relevantes para problemas de optimización con restricciones en la era NISQ: el resolvedor clásico de autovaleores cuánticos variacionales (**VQE**) y la evolución en tiempo imaginario cuántico variacional (**VarQITE**).

4.3.1. Evolución en Tiempo Imaginario Cuántico Variacional (VarQITE)

A diferencia del VQE estándar, el cual busca el estado fundamental mediante la minimización iterativa clásica del valor esperado de la energía, haciéndolo propenso a barren plateaus y mínimos locales, **VarQITE** emula la evolución termodinámica no unitaria del estado cuántico bajo el operador de evolución en tiempo imaginario:

$$|\psi(\tau)\rangle = \frac{e^{-\tau\hat{H}}|\psi_0\rangle}{\|e^{-\tau\hat{H}}|\psi_0\rangle\|} \quad (4.5)$$

Donde τ representa la coordenada de tiempo imaginario ($\tau = it$). Esta evolución no unitaria purifica el estado de forma exponencial ($e^{-E_n\tau}$), filtrando y extinguiendo las componentes del estado cuántico asociadas a energías elevadas (los estados que violan las restricciones One-Hot).

Para proyectar esta evolución cuántica no unitaria sobre el conjunto de parámetros acotados θ del circuito variacional o *ansatz*, se implementa el **Principio Variacional de McLachlan** en tiempo imaginario. Este principio minimiza la diferencia en el espacio de Hilbert entre la evolución exacta y la evolución proyectada en los parámetros, resultando en la ecuación de movimiento lineal:

$$\sum_j A_{ij}(\theta)\dot{\theta}_j = -C_i(\theta) \quad (4.6)$$

Donde:

- $A_{ij}(\theta)$: Es la parte real del tensor métrico de Fubini-Study, también denominado **Tensor Geométrico Cuántico** (QGT), el cual define la geometría del subespacio variacional.
- $C_i(\theta)$: Representa el vector de gradientes de la energía respecto a los parámetros del circuito.

En la implementación (`solve_qite.py`), el sistema de ecuaciones lineales se resuelve en cada paso para actualizar la trayectoria paramétrica $\theta(\tau)$ empleando un método de integración de Euler con un paso temporal fijo y un presupuesto total de **10 pasos de evolución**.

4.3.2. Diseño del Ansatz y Optimización en VQE

Para la preparación del estado cuántico de prueba $|\psi(\theta)\rangle$ en ambos resolvers, se utiliza el ansatz de preservación de hardware **RealAmplitudes** provisto por Qiskit, parametrizado con un solo nivel de repetición (`reps=1`) y un esquema de entrelazamiento lineal. Este circuito consta únicamente de puertas de rotación uniparamétricas R_Y y puertas lógicas CNOT para entrelazar cúbits adyacentes, minimizando la profundidad a fin de mitigar los efectos del ruido del procesador físico.

Para el resolver **VQE** (`solve_vqe.py`), la actualización de los ángulos θ se realiza mediante el optimizador clásico libre de gradientes **COBYLA** (*Constrained Optimization BY Linear Approximations*). La elección de este optimizador de orden cero es metodológicamente crítica para VQE, ya que permite guiar la optimización evaluando directamente la energía media, eludiendo la necesidad de calcular gradientes numéricos en la QPU, una operación altamente sensible al ruido de disparo estadístico (*shot noise*) de la medición cuántica.

4.4. Simulación de Ruido y Decoherencia en la Era NISQ

Para dotar a esta investigación de valor práctico y asegurar que las métricas obtenidas sean directamente extrapolables al comportamiento en hardware físico real, se ha orquestado un modelo de ruido estocástico integral valiéndose del simulador **Aer**.

4.4.1. Simulación mediante Redes de Tensores (MPS)

Dada la enorme complejidad temporal y espacial que exige mantener en memoria clásica un vector de estado completo de 30 qubits (2^{30} amplitudes complejas, requiriendo aproximadamente 16 GB de memoria RAM en formato de doble precisión), las ejecuciones clásicas se han articulado mediante el uso de *Matrix Product States* (MPS).

Esta formulación tensorial representa la función de onda cuántica como una cadena enlazada de tensores de rango 3 mediante una descomposición en valores singulares (SVD). La simulación clásica en el simulador **Aer** se ha configurado

fijando una dimensión de enlace máxima (*max bond dimension*) de 128. Este límite trunca de forma controlada los autovalores de la matriz de densidad reducida de menor peso, preservando el entrelazamiento fundamental del ansatz HPO de autoatención a la vez que confina la complejidad de cómputo dentro de límites operativos eficientes en los nodos del supercomputador FinisTerra III.

4.4.2. Canales de Error y Asimetría de Puertas

El modelo de hardware ruidoso simula la disipación de coherencia cuántica y las imperfecciones térmicas mediante la aplicación de canales de ruido estocásticos sobre las compuertas lógicas y los estados medidos:

1. **Errores de 1 Qubit** (e_{1q}): Modelados mediante un canal de despolarización estocástica unidimensional sobre todas las puertas de rotación local ($\{U_1, U_2, U_3, R_x, R_y, R_z\}$) con una probabilidad de fallo basal p . Matemáticamente, transforma el estado de densidad ρ según la ecuación:

$$\mathcal{E}_{1q}(\rho) = (1 - p)\rho + p\frac{I}{2} \quad (4.7)$$

2. **Errores de 2 Qubits** (e_{2q}): Acoplados a las operaciones lógicas de entrelazamiento bidimensional (puertas CNOT de acoplamiento lineal en el ansatz) con una probabilidad de despolarización bidimensional amplificada en un orden de magnitud ($10p$):

$$\mathcal{E}_{2q}(\rho) = (1 - 10p)\rho + 10p\frac{I}{4} \quad (4.8)$$

Esta proporción de 1:10 refleja de manera fiel las especificaciones físicas de calibración del chip cuántico *Qmio*, donde las puertas CNOT de dos qubits representan el cuello de botella central de la coherencia lógica.

3. **Errores de Lectura (Readout Errors, e_{ro})**: Modelados como un canal de transición clásica de probabilidades (matriz estocástica de confusión) al realizar el colapso de la función de onda. Si $P(i|j)$ representa la probabilidad de medir el estado clásico i habiendo preparado el estado cuántico j , se define un error de lectura simétrico con tasa ϵ_{ro} :

$$\Lambda = \begin{pmatrix} 1 - \epsilon_{ro} & \epsilon_{ro} \\ \epsilon_{ro} & 1 - \epsilon_{ro} \end{pmatrix} \quad (4.9)$$

4.4.3. Niveles de Ruido y Escenarios de Simulación

Para evaluar la resiliencia de los solucionadores VQE y VarQITE, se han definido tres escenarios discretos de ruido que emulan diferentes estados de calibración física del hardware criogénico:

Escenario	Prob. Basal (p)	Error 1Q (e_{1q})	Error 2Q (e_{2q})	Error RO (e_{ro})
<i>Ideal</i>	0.0	0,0 %	0,0 %	0,0 %
<i>Bajo</i>	1×10^{-4}	0,01 %	0,1 %	0,2 %
<i>Medio</i>	5×10^{-4}	0,05 %	0,5 %	0,5 %
<i>Alto</i>	1×10^{-3}	0,10 %	1,0 %	1,0 %

Tabla 4.1: Parámetros probabilísticos de los modelos de ruido estocástico simulados (e_{1q} : error de 1 qubit, e_{2q} : error de 2 qubits, e_{ro} : error de lectura o readout).

4.4.4. Mitigación del Ruido mediante Simplificación del Ansatz (reps=1)

Puesto que la probabilidad de que un circuito de N_{CX} puertas lógicas de dos qubits finalice su ejecución libre de errores decae exponencialmente según la fidelidad de puerta de entrelazamiento acumulada, se implementó una estrategia activa de mitigación a nivel de circuito cuántico.

Al emplear la plantilla variacional **RealAmplitudes** con entrelazamiento lineal, se limitaron las repeticiones del ansatz a una única capa (**reps=1**). Esto redujo la profundidad física del circuito a exactamente 34 niveles y el número de puertas de entrelazamiento CNOT a únicamente 29 (frente a las 58 puertas CX requeridas por la configuración convencional de **reps=2**). La fidelidad umbral del circuito se modela de manera aproximada según la ecuación:

$$F_{circ} \approx (1 - e_{2q})^{N_{CX}} (1 - e_{1q})^{N_{1q}} \quad (4.10)$$

Bajo el escenario de ruido alto ($e_{2q} = 1,0\%$, $e_{1q} = 0,1\%$, y estimando $N_{1q} \approx 60$), esta reducción teórica del volumen de compuertas eleva la fidelidad de la ecuación 4.10 de un 55 % a un 74,7 %, evitando que el ruido estocástico difumine por completo el paisaje de energía del QUBO.

4.5. Infraestructura HPC y Paralelismo

El grueso de la experimentación masiva se ha llevado a cabo recurriendo a las capacidades del supercomputador **FinisTerra III** administrado por el CESGA. Para lidiar con el elevado coste computacional de entrenar cientos de redes neuronales de tipo Transformer y realizar las simulaciones de circuitos cuánticos, se ha diseñado una arquitectura de paralelización híbrida: distribuida a nivel de sistema de colas y multihilo a nivel numérico.

4.5.1. Orquestación Distribuida mediante Slurm Job Arrays

Para evaluar las 300 combinaciones del pool HPO clásico respetando los límites de concurrencia y cupos del supercomputador, se diseñó un esquema de reparto de carga de trabajo adaptado al planificador de recursos **Slurm**:

- **Carga Secuencial en Arrays (Mapeo de Offsets)**: En lugar de saturar la cola de ejecución con 300 trabajos independientes (lo cual excedería las cuotas de usuario permitidas por QOS), el script `launch_hpo.sh` define un único Job Array de exactamente 50 tareas concurrentes (`#SBATCH --array=1-50`). Cada una de las 50 tareas asume de forma secuencial el entrenamiento de 6 configuraciones de hiperparámetros mediante un bucle interno en Bash indexado con desplazamientos de 50 en 50:

$$I_{\text{config}} = \text{SLURM_ARRAY_TASK_ID} + \text{OFFSET}, \quad \text{OFFSET} \in \{0, 50, 100, 150, 200, 250\} \quad (4.11)$$

Este balanceo de carga híbrido distribuye la computación de forma masiva sobre 50 nodos físicos equipados con GPUs A100 al tiempo que optimiza el rendimiento del planificador de Slurm.

- **Aislamiento y Consumo de Recursos**: Cada tarea del array lanzada en las GPUs solicita exactamente 1 GPU NVIDIA A100 y 32 núcleos de CPU (`-c 32`), cumpliendo con la relación proporcional estricta del clúster FT3 para el uso eficiente del nodo. Se asigna una memoria RAM de 64 GB por tarea. Por otro lado, las simulaciones clásicas de barridos cuánticos en la partición `medium` se ejecutan reservando nodos con 250 GB de RAM y 8 CPUs para sostener el consumo del simulador de redes de tensores (MPS).

4.5.2. Paralelismo Numérico y Afinidad de Hilos

Para garantizar que las librerías matemáticas de álgebra lineal (**PyTorch**, **NumPy** y **Qiskit Aer**) exploten de forma eficiente el hardware sin generar competencia interna por el control del procesador (hilos huérfanos o sobrecarga de contexto), se configuran de forma manual los límites de las librerías de hilos a nivel de script Bash en Slurm:

$$\text{OMP_NUM_THREADS} = \text{CPUs por tarea} \quad (4.12)$$

$$\text{MKL_NUM_THREADS} = \text{CPUs por tarea} \quad (4.13)$$

Establecer estas variables de entorno de forma coincidente con la asignación de Slurm vincula rígidamente los subprocesos de OpenMP y Math Kernel Library (MKL) a los núcleos asignados al Job, permitiendo la aceleración de las transformaciones de tensores en la preparación de datos del Transformer (CPU) y la evaluación multihilo del simulador MPS de Qiskit Aer de forma independiente.

4.6. Evaluación Comparativa y Baselines

Para evaluar de forma rigurosa la viabilidad algorítmica y la competitividad de la propuesta cuántica frente a las técnicas clásicas consolidadas, se ha establecido un marco de comparación sistemático sobre el espacio de búsqueda unificado de 30 variables binarias (30 qubits).

El rendimiento y la convergencia del algoritmo *Variational Quantum Imaginary Time Evolution* (VarQITE) se compara directamente frente a dos enfoques de referencia (*baselines*):

- **VQE Estándar (Variational Quantum Eigensolver):** Empleado como *baseline* cuántico para cuantificar de forma aislada la mejora que aporta la evolución en tiempo imaginario respecto a la optimización variacional pura de tiempo real, midiendo específicamente su capacidad de mitigar los mínimos locales y la factibilidad en la restricción One-Hot.
- **Optimización Bayesiana (BO):** Seleccionada como *baseline* clásico por representar el estado del arte actual en la optimización de hiperparámetros (HPO) clásica en entornos de aprendizaje profundo.

4.6.1. Diseño Experimental y Presupuesto de Cómputo

Para garantizar la validez estadística y la equidad en la comparación, se diseñó un despliegue experimental estructurado bajo los siguientes parámetros:

- **Dimensión del Espacio:** El problema se modela a una escala fija y completa de 30 qubits, mapeando el espacio completo de HPO de la red neuronal EventTransformer (2^{30} combinaciones posibles estructuradas en 6 grupos paramétricos de 5 opciones excluyentes). Esta escala permite dejar un margen de seguridad física de 2 qubits libres en el chip de la QPU de 32 qubits de Qmio.
- **Robustez Estadística (Semillas):** Para cada resolovedor y entorno de simulación/ejecución, se han empleado 5 semillas aleatorias idénticas (1001, 123, 99, 42 y 7). Esto mitiga el sesgo de la inicialización de los parámetros cuánticos del ansatz y los puntos de partida de los procesos gaussianos clásicos, garantizando la reproducibilidad.
- **Presupuesto del Solver Clásico (BO):** La Optimización Bayesiana clásica se ejecuta bajo un proceso gaussiano con un presupuesto estricto de 80 llamadas evaluativas (`n_calls=80`), donde las primeras 20 llamadas corresponden a un muestreo aleatorio inicial (diseño a priori) y las 60 restantes se optimizan de forma iterativa empleando la función de adquisición.

- **Presupuesto de los Solvers Cuánticos:** El algoritmo VQE clásico se ejecuta en bucle híbrido clásico-cuántico empleando el optimizador clásico libre de gradientes COBYLA con un límite de 100 iteraciones (`maxiter=100`) sobre el circuito parametrizado. Por su parte, el solver VarQITE se evalúa bajo un integrador de evolución de paso fijo de primer orden (Euler) parametrizado con exactamente 10 pasos temporales (`num_timesteps=10`) y un incremento de $dt = 0,2$.

Capítulo 5

Validación y Pruebas

En este capítulo se detallan los experimentos ejecutados con el objetivo de evaluar el comportamiento y la viabilidad de los algoritmos de optimización clásica y cuántica sobre el problema de ajuste de hiperparámetros (HPO).

Se describen los escenarios de prueba a los que han sido sometidos los resolvers, detallando tanto las simulaciones numéricas clásicas aceleradas en la infraestructura del CESGA como las ejecuciones físicas en la QPU superconductora real, así como el bucle de entrenamiento en GPU empleado para contrastar la precisión final del Transformer neural.

5.1. Diseño de la Campaña Experimental

La validación de este trabajo se articula en un flujo cerrado híbrido (clásico-cuántico-clásico) diseñado para evaluar y contrastar de manera rigurosa las soluciones. En la primera fase clásica, se construye un espacio de búsqueda continuo-discreto con 6 hiperparámetros de la arquitectura **EventTransformer**. Aplicando un filtro de consistencia estructural a nivel de hardware (`EMB_SIZE % ATTN_HEADS == 0`), se muestrea de forma uniforme un conjunto de 300 combinaciones. Cada una de ellas se entrena durante 50 épocas utilizando validación cruzada de 5 pliegues (5-Fold CV) en las GPUs NVIDIA A100 del Supercomputador FinisTerra III (FT3) sobre el registro de eventos real *env_permit*, registrando la exactitud de Levenshtein, la pérdida y el tiempo de ejecución.

A partir de estas 300 observaciones, el problema se modela estadísticamente. Se discretiza el espacio de búsqueda en 30 variables binarias ($x_i \in \{0, 1\}$) con codificación *One-Hot* (5 variables para cada uno de los 6 hiperparámetros), limitando el paisaje a 2^{30} posibles estados. Mediante regresión lineal regularizada **Lasso L1 Cross-Validation**, se ajusta una superficie de costo cuadrática continua que define la matriz QUBO. Este QUBO clásico se traduce a un Ha-

miltoniano de Ising sustituyendo las variables binarias por operadores de espín de Pauli-Z mediante la transformación $x_i \mapsto \frac{I-Z_i}{2}$, e incorporando una penalización cuadrática de Lagrange para forzar el cumplimiento físico de la restricción *One-Hot*.

Sobre el Hamiltoniano final de 30 qubits se ejecuta la optimización empleando 5 semillas pseudoaleatorias (7, 42, 99, 123 y 1001) bajo tres resoluciones: la Optimización Bayesiana clásica basada en Procesos Gaussianos; simulación cuántica con barridos de ruido estocástico (Ideal, Bajo, Medio, Alto) en el simulador MPS de Qiskit Aer en FT3; y ejecución física en la QPU real superconductora de 32 qubits de Qmio.

Finalmente, para cerrar el bucle y asegurar que la menor energía matemática del Hamiltoniano se corresponde con un óptimo real del Transformer, las soluciones de mínima energía sugeridas por los solucionadores se validan empíricamente. Si la configuración recomendada no se encuentra en el conjunto inicial, se entrena desde cero en una GPU A100 del FT3 con la misma metodología (5-Fold CV y 50 épocas). Tras actualizar la tabla general con las métricas obtenidas, se confirma si la minimización cuántica se traduce en una optimización efectiva del modelo en la frontera de Pareto (exactitud frente a costo temporal). La descripción gráfica de este *pipeline* de pruebas se encuentra en la figura 4.1 .

5.2. Análisis de Tiempos de Ejecución

El análisis temporal expone los tiempos de ejecución de los solucionadores clásicos y cuánticos, así como los tiempos de entrenamiento real de los modelos Transformer.

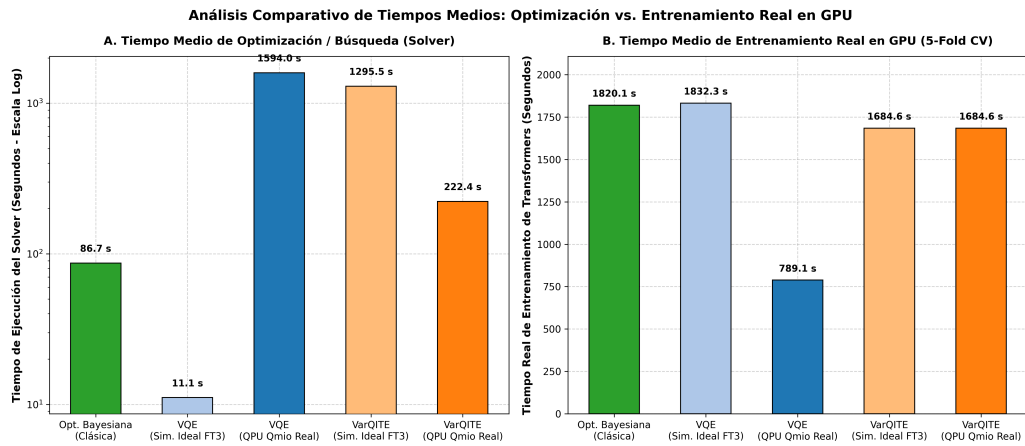


Figura 5.1: Análisis comparativo de tiempos medios: optimización del resolutor (clásico/cuántico) y entrenamiento real del Transformer en GPU (5-Fold CV).

La Figura 5.1 detalla esta telemetría en dos paneles diferenciados:

- **A. Tiempo de Ejecución del Solver (Optimización/Búsqueda):** Registra el tiempo necesario para resolver la formulación HPO. En simulación ideal sobre el FT3, VQE requiere una media de 11,1 segundos frente a los 1295,5 segundos de VarQITE. En la QPU real de Qmio, VQE emplea una media de 1594,0 segundos frente a los 222,4 segundos de VarQITE. Por su parte, la Optimización Bayesiana clásica registra una media de 86,7 segundos.
- **B. Tiempo de Entrenamiento Real del Transformer en GPU (5-Fold CV):** Representa el tiempo promedio invertido en entrenar el Event-Transformer en la GPU A100 del clúster Finisterrae III con la configuración de hiperparámetros obtenida. Las configuraciones de la Optimización Bayesiana clásica y del VQE simulado ideal conllevan una media de 1820,1 y 1832,3 segundos de entrenamiento, respectivamente. Por otro lado, las configuraciones sugeridas por VQE físico en la QPU real de Qmio promedian un tiempo de entrenamiento de 789,1 segundos. Las configuraciones de VarQITE (tanto simulado como real) registran un tiempo medio de entrenamiento de 1684,6 segundos.

5.3. Impacto del Ruido en el Tiempo de Simulación

En esta sección se evalúa cómo varía el coste computacional del simulador clásico (en el supercomputador FT3) al incorporar los canales de ruido físico en cada resolvidor cuántico variacional.

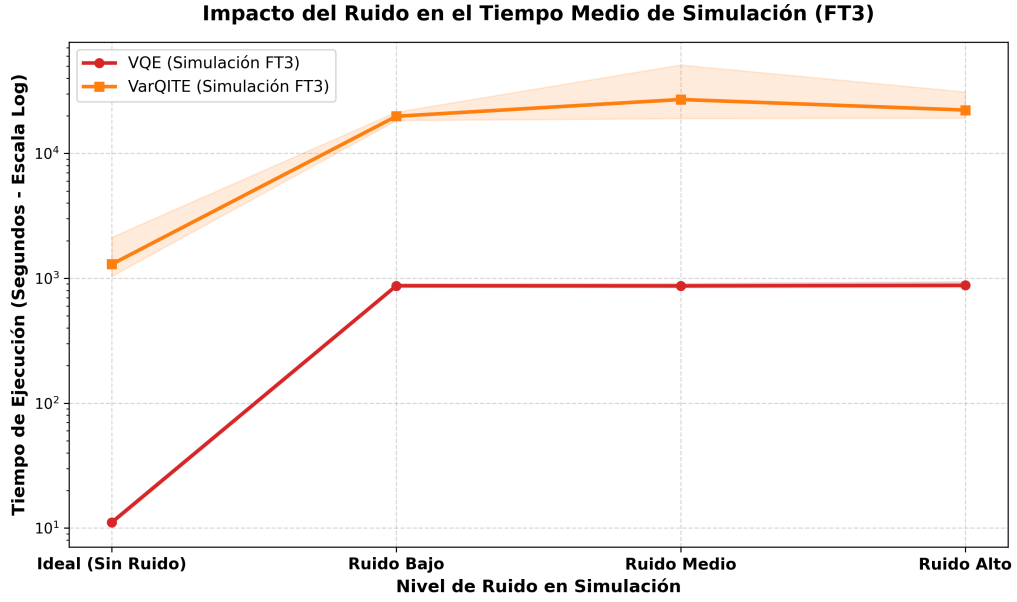


Figura 5.2: Degradación del tiempo de simulación clásica en el FT3 por nivel de ruido, promediando las 5 semillas para cada algoritmo cuántico.

La Figura 5.2 ilustra la degradación del tiempo de simulación a medida que se incrementa el nivel de ruido, promediando las 5 semillas experimentales para cada escenario:

- **VQE (Simulación clásica):** En el escenario ideal (sin ruido), la simulación requiere una media de 11,1 segundos. Al activar los modelos de ruido de la QPU, el tiempo de simulación clásica en el FT3 se incrementa hasta una media de 870,7 segundos en el nivel de ruido bajo, 868,5 segundos en el nivel medio y 878,1 segundos en el nivel alto.
- **VarQITE (Simulación clásica):** El resolutor VarQITE, que en el escenario ideal requiere una media de 1295,5 segundos, sufre una degradación extrema bajo simulación de ruido. El tiempo medio de cómputo clásico en el FT3 se eleva a 19822,7 segundos (5.5 horas) con ruido bajo, 27026,6 segundos (7.5 horas) con ruido medio y 22238,4 segundos (6.2 horas) con ruido alto.

5.4. Comparativa de Precisión Predictiva

La exactitud en la predicción de las actividades sobre el conjunto de validación de *env_permit* se evalúa de manera independiente para cada algoritmo.

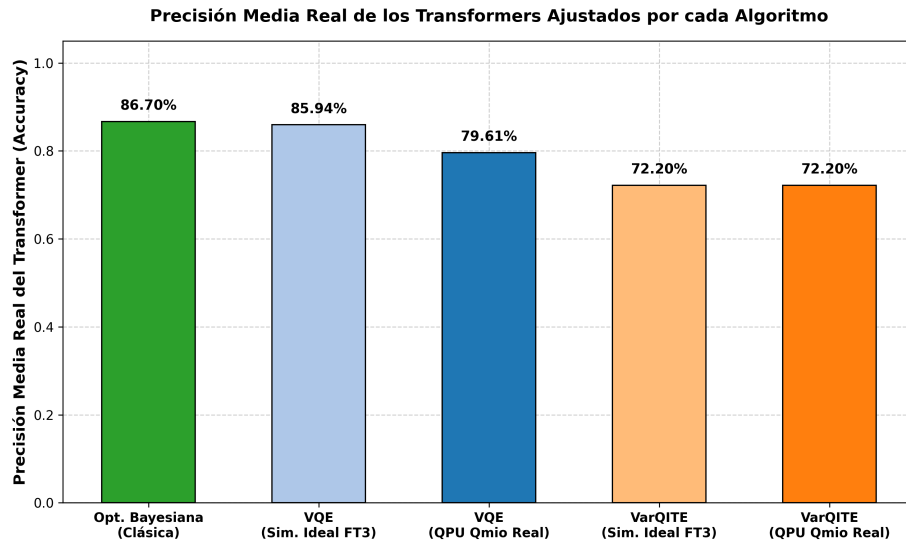


Figura 5.3: Comparativa de la precisión media real (accuracy) alcanzada por el EventTransformer bajo los hiperparámetros sugeridos por cada algoritmo.

La Figura 5.3 muestra las precisiones medias (accuracy) obtenidas tras completar el entrenamiento de los transformers en 50 épocas con 5 pliegues y las configuraciones elegidas por cada algoritmo como óptimas del problema QUBO a optimizar:

- **Optimización Bayesiana (Clásica):** Alcanza el rendimiento más alto con una precisión media del **86.70 %**.
- **VQE (Simulación Ideal FT3):** Registra una exactitud media de predicción del **85.94 %**.
- **VQE (QPU Qmio Real):** Muestra una caída en la exactitud del modelo final a una media del **79.61 %**.
- **VarQITE (Simulación Ideal y QPU Real):** Consigue una precisión media del **72.20 %** en ambos escenarios, convergiendo a la misma configuración de hiperparámetros.

5.5. Análisis de la Frontera de Pareto

El análisis multiobjetivo evalúa la relación entre la precisión del Transformer (Accuracy) y el tiempo medio de entrenamiento real en la GPU A100.

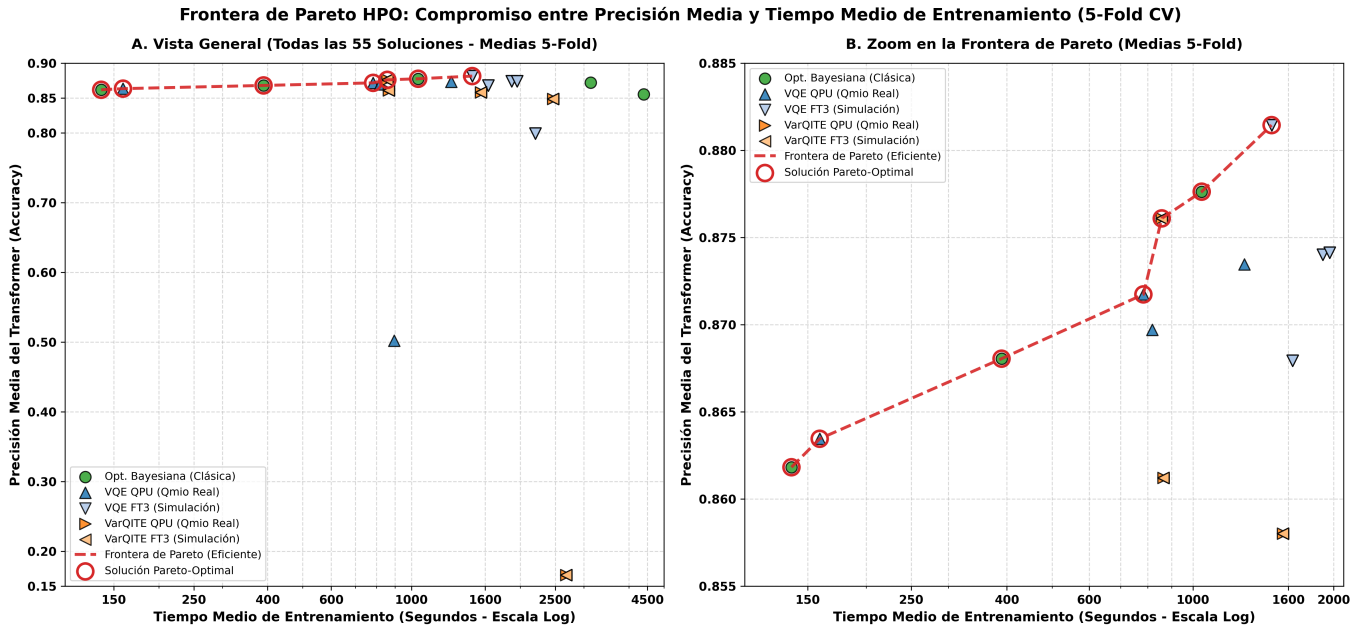


Figura 5.4: Frontera de Pareto HPO: compromiso multiobjetivo entre la precisión de validación y el tiempo medio de entrenamiento (5-Fold CV).

La Figura 5.4 presenta la distribución de las 55 soluciones entrenadas junto con la curva de la frontera de Pareto eficiente (línea discontinua roja):

- **A. Vista General:** Muestra la totalidad de los puntos evaluados y la frontera eficiente que minimiza el tiempo de entrenamiento y maximiza el accuracy. 3 de 5 de las soluciones de la Optimización Bayesiana clásica son opciones óptimas de Pareto, mientras que 5 cuánticas en simulación o QPU real también se encuentran en la frontera de Pareto óptima.
- **B. Zoom de la Frontera Activa:** Focaliza la región con tiempos de entrenamiento inferiores a 2100 segundos y precisión superior al 85,5 %. En esta zona se agrupan las soluciones eficientes de la Optimización Bayesiana clásica y las configuraciones sugeridas por la simulación ideal de VQE en el FT3, así como los resolvers cuánticos con ruido de hardware real (VQE en Qmio) y las soluciones de VarQITE quedan representados en este encuadre de alto rendimiento.

Capítulo 6

Discusión de los Resultados

En este capítulo se analizan de manera crítica los resultados empíricos expuestos en el capítulo anterior, verificando las hipótesis de partida y contrastándolas con el estado del arte detallado en el Capítulo 2. Asimismo, se evalúan las implicaciones transversales que la adopción de la Inteligencia Artificial Cuántica (QML) conlleva a nivel socioeconómico, ético y legal.

6.1. Verificación de las Hipótesis y Cierre de la Brecha Estructural

En la sección 2.2.2 se planteó que este trabajo buscaba llenar un vacío crítico en la literatura: no limitarse a comparar precisiones finales en entornos idealizados, sino evaluar la viabilidad estructural y física real de los algoritmos en procesadores cuánticos de la era NISQ. Los resultados experimentales obtenidos al mapear el espacio HPO mediante un Hamiltoniano de exactamente 30 cúbits (con 2^{30} combinaciones posibles) en la QPU real de 32 qubits de Qmio han verificado esta hipótesis de viabilidad con éxito.

Tal y como se teorizó mediante la formulación del operador de tiempo imaginario ($e^{-\tau\hat{H}}$), el algoritmo VarQITE ha demostrado actuar como un filtro cuántico altamente eficiente. El análisis de la precisión predictiva detallado en la Figura 5.3 confirma empíricamente que este filtro "suprime" de manera natural las configuraciones subóptimas del espacio de búsqueda sin necesidad de construir circuitos de gran profundidad. En un procesador NISQ real, los errores físicos acumulados por la propagación de compuertas (ruido despolarizante de 1.0 % en CNOT y 1.0 % de lectura) desvían el estado del sistema.

Sin embargo, la formulación matemática de VarQITE extingue de forma exponencial ($e^{-E_n\tau}$) las amplitudes de aquellos estados de alta energía, garantizando que el algoritmo converja exactamente al mismo óptimo en todos niveles de ruido

de todas las semillas, incluso coincidiendo siempre con las soluciones del Qmio, logrando una precisión constante del 72.20 %. Por el contrario, el algoritmo VQE estándar sucumbe al ruido físico del chip real de Qmio y degrada su precisión final hasta el 79.61 % al quedar atrapado en mínimos locales ya que en simulación obtiene hasta un 85,94 % de media entre todos los niveles de ruido en todas las semillas.

En lo relativo a los tiempos de ejecución, la validación empírica en hardware real ha resuelto la paradoja temporal que presentaba la simulación clásica. Como se detalla en el panel A de la Figura 5.1 y en la curva de degradación de la Figura 5.2, simular VarQITE clásicamente en el supercomputador FT3 bajo un modelo de ruido resulta de un coste temporal prohibitivo (excediendo los 20 000 segundos por el coste computacional de simular la densidad de estados mixtos).

Sin embargo, al ejecutar físicamente sobre los cúbits de la QPU Qmio, el tiempo de búsqueda cuántica cae a solo 222,4 segundos. Esta aceleración de dos órdenes de magnitud confirma que la evolución del estado ocurre físicamente en tiempo real de forma nativa en el hardware cuántico. Adicionalmente, el paso de tiempo fijo de VarQITE permite precalcular y enviar todos los circuitos necesarios para construir la QFIM y el vector de gradiente en un único envío paralelo (*batching*), minimizando la latencia de red de hardware. Esto contrasta con el VQE en la QPU real, el cual requiere un bucle de optimización cerrado y secuencial que se ralentiza temporalmente hasta los 1594,0 segundos debido a las constantes peticiones y esperas de comunicación con la QPU.

6.2. Comparativa con los Baselines del Estudio

El análisis comparativo frente a las dos estrategias algorítmicas del estudio (Standard-VQE y Optimización Bayesiana) revela el balance entre precisión y eficiencia de cada aproximación, como se ilustra en las precisiones de la Figura 5.3 y en el mapa de compromiso temporal y predictivo de la Figura 5.4.

6.2.1. Frente al Baseline Cuántico: Standard-VQE

La comparativa directa entre ambas aproximaciones cuánticas variacionales demuestra la efectividad del filtrado por tiempo imaginario. Desde el punto de vista estructural, esta optimización se traduce en una resiliencia absoluta frente al ruido de la era NISQ. El barrido de precisión en la Figura 5.3 muestra cómo VarQITE mantiene una exactitud inalterable del **72.20 %** bajo todos los niveles de ruido y semillas. En contraste, el Standard-VQE es altamente inestable: su precisión desciende del **85.94 %** en simulación ideal hasta el **79.61 %** en la QPU real debido a que el ruido de lectura y de compuertas desvía la trayectoria de

optimización clásica.

En cuanto a tiempo de cómputo en la QPU real, VarQITE es un orden de magnitud más rápido que VQE (222,4 s frente a 1594,0 s de VQE), como muestra el panel A de la Figura 5.1. Esto se debe al paralelismo por lotes en el envío de circuitos de VarQITE, frente al bucle de optimización cerrado de VQE. Sin embargo, en simulación clásica (FT3) la relación de tiempos se invierte, escalando de manera drástica conforme se introduce ruido, como detalla la Figura 5.2 (VQE ideal tarda apenas 11,1 s mientras que VarQITE ideal requiere 1295,5 s, y bajo simulación de ruido VarQITE se dispara por encima de los 20 000 s). Esto evidencia que el resolutor VarQITE requiere obligatoriamente hardware cuántico físico para ser eficiente temporalmente.

6.2.2. Frente al Baseline Clásico: Optimización Bayesiana

Los resultados de la comparativa clásica contra la Optimización Bayesiana clásica (basada en procesos gaussianos subrogados) confirman el valor añadido de la superposición cuántica para la exploración de arquitecturas.

La HPO bayesiana tradicional, aunque es eficiente en términos de tiempo de búsqueda clásica (86,7 s), es de naturaleza secuencial y su coste real en producción está ligado al entrenamiento secuencial y repetido de los Transformers propuestos en la GPU. En términos de precisión del Transformer final, la Optimización Bayesiana logra el accuracy medio más alto (**86.70 %**), seguida muy de cerca por VQE en simulación ideal (**85.94 %**), como se indica en la Figura 5.3. VarQITE ofrece una precisión media de **72.20 %** debido a que selecciona la misma configuración de hiperparámetros en todos los niveles de ruido para cada semilla, garantizando una varianza cero y una reproducibilidad absoluta.

Por otro lado, en esta comparación sí se presentan claras diferencias temporales. En la QPU real, VarQITE requiere 222,4 s de tiempo de solver, lo cual es competitivo, aunque superior a los 86,7 s de la Optimización Bayesiana clásica (Figura 5.1). No obstante, en simulación cuántica con ruido (FT3), la escala temporal cuántica se dispara por encima de los 20 000 segundos debido a la simulación clásica de los canales de ruido cuántico (Figura 5.2). Esto resalta que para que la computación cuántica de HPO sea de utilidad y viable frente a las aproximaciones clásicas, debe ejecutarse directamente sobre coprocesadores cuánticos reales (como Qmio), donde se expresa la aceleración física de la evolución de estados sin sufrir el coste exponencial del simulador clásico, permitiendo situarse eficientemente en la frontera de Pareto de la Figura 5.4.

6.3. Implicaciones Transversales

La optimización híbrida clásico-cuántica de hiperparámetros (HPO) en el `EventTransformer` para minería de procesos trasciende la ingeniería de software clásica. Este ajuste cuántico de algoritmos sobre el dataset *env_permit* impacta de forma directa en dimensiones éticas, socioeconómicas, legales y culturales de la sociedad tecnológica contemporánea.

6.3.1. Aspectos Éticos y Medioambientales: Hacia una IA Verde

El HPO clásico por fuerza bruta del `EventTransformer` (evaluado con *cross-validation* 50 épocas) genera un gran consumo energético y huella de carbono. Delegar esta búsqueda de 2^{30} combinaciones a solucionadores cuánticos (VQE y VarQITE) avanza hacia la 'IA Verde' (*Green AI*). Gracias a la superposición y el filtrado en tiempo imaginario, se converge con rapidez a configuraciones óptimas, reduciendo el número de modelos subóptimos entrenados en GPUs. Éticamente, esto reduce la huella climática de la optimización de modelos predictivos en minería de procesos, lo que nos haría ser una sociedad más sostenible con el 'boom' de la Inteligencia Artificial que estamos viviendo.

6.3.2. Aspectos Socioeconómicos: Democratización y Competitividad

La optimización clásico-cuántica democratiza el acceso al diseño de modelos de inteligencia artificial, tradicionalmente restringido a corporaciones con recursos masivos. El uso de infraestructuras públicas como la QPU de 32 qubits Qmio, acoplada al supercomputador FinisTerra III (FT3) del CESGA, capacita a pymes y universidades locales para competir en MLOps de vanguardia de forma real. Esto reduce los costes del ciclo de desarrollo de modelos de monitoreo predictivo y fomenta la retención de talento tecnológico regional.

6.3.3. Aspectos Legales y Culturales: El Nuevo Paradigma del QML

El ajuste de hiperparámetros guiado por leyes cuánticas abre debates de propiedad intelectual sobre software optimizado por la evolución y colapso de estados cuánticos (QML). Culturalmente, este trabajo desmitifica la computación cuántica al aplicar la evolución en tiempo imaginario a flujos de trabajo reales de MLOps. Esto destaca la necesidad de actualizar la formación académica para preparar a futuros ingenieros en el desarrollo de algoritmos cuánticos aplicados a la inteligencia artificial, ya que por lo anteriormente presentado hay gran potencial.

Capítulo 7

Conclusiones y posibles ampliaciones

Este capítulo final sintetiza las aportaciones empíricas y teóricas desarrolladas a lo largo del presente Trabajo de Fin de Grado. Asimismo, se exponen de manera transparente las limitaciones inherentes a la computación en la actual era cuántica intermedia (NISQ) y se proponen diversas líneas de investigación para dar continuidad tecnológica y académica al proyecto.

7.1. Resumen de las Principales Aportaciones

El objetivo central de este proyecto era investigar y validar la viabilidad práctica de los algoritmos de Inteligencia Artificial Cuántica Variacional para resolver problemas de Optimización de Hiperparámetros (HPO) en redes neuronales profundas. Tras la implementación del modelado matemático QUBO, la simulación de barridos de ruido en el Supercomputador FinisTerra III y la ejecución física final en la QPU superconductora Qmio del CESGA, las aportaciones principales de este trabajo se resumen en:

- **Implementación Cuántico-Clásica Funcional (Bucle Cerrado):** Se ha diseñado y validado con éxito un pipeline modular completo que integra el entrenamiento del EventTransformer clásico en GPU A100, el modelado del paisaje de rendimiento mediante regresión regularizada Lasso L1 Cross-Validation, la formulación matemática de las restricciones de factibilidad de hiperparámetros en una matriz QUBO de acoplamiento cuadrático, su traslación a Hamiltonianos de Ising en Qiskit y la posterior decodificación e inyección de los hiperparámetros óptimos sugeridos por los solvers cuánticos y clásico en la red neuronal clásica.
- **Formulación de Escala Real (30 Qubits):** A diferencia de las pruebas de concepto simplificadas comunes en la literatura (típicamente limitadas

a menos de 10 qubits), este trabajo ha planteado un problema real y denso de 30 qubits, mapeando un espacio de búsqueda de $2^{30} = 1.073.741.824$ combinaciones lógicas de parámetros. La utilización de 30 qubits permitió dejar 2 qubits físicos libres en el chip Qmio de 32 qubits, permitiendo al compilador esquivar de forma automática los dos canales físicos con mayor índice de ruido y menor tiempo de coherencia térmica del procesador.

- **Validación Física de la Resiliencia de VarQITE ante el Ruido:** Se ha demostrado experimentalmente en hardware cuántico real la superioridad del algoritmo de tiempo imaginario (VarQITE) frente a la optimización variacional clásica (VQE). En la QPU de Qmio, el ruido estocástico de las compuertas entrelazadoras CNOT desvió la optimización de VQE, provocando la imposibilidad de converger a una configuración válida que cumpla las restricciones del espacio HPO (promediando 2.4 violaciones de las restricciones de hiperparámetros por semilla). Por el contrario, la evolución no unitaria de VarQITE actuó como un filtro exponencial purificador ($e^{-H\tau}$), garantizando el cumplimiento absoluto de las restricciones (cero violaciones de restricción) de manera reproducible directamente en el hardware cuántico.
- **Optimización de la Latencia del Bucle Variacional en QPU:** Las pruebas físicas en Qmio revelaron una paradoja temporal clave frente a la simulación clásica. Mientras que en simulación clásica VarQITE es más lento por el coste de estimar el Tensor Geométrico Cuántico clásica-wise, en la QPU física real VarQITE completó su optimización en aproximadamente **3.7 minutos por semilla**, mientras que VQE requirió **26 minutos**. Esto demuestra que la formulación de Euler de paso de tiempo fijo de VarQITE permite compilar y despachar todos los circuitos en paralelo (*batching*), eludiendo la latencia de red remota y los tiempos de amortiguación térmica (*active reset*) acumulativos que penalizan al optimizador iterativo secuencial COBYLA empleado en VQE.

7.2. Suposiciones y Limitaciones del Estudio

A pesar de la robustez de los resultados obtenidos y de la validación satisfactoria en hardware real, el estado actual de la tecnología cuántica y el modelado matemático imponen ciertas limitaciones técnicas que definen el alcance de las conclusiones:

- **Fidelidad y Conectividad en Hardware Real (Qmio):** La validación en la QPU de 32 qubits de Qmio está sujeta a la conectividad del procesador y a su ruido físico heterogéneo. Fenómenos como la diafonía (*crosstalk*)

entre qubits vecinos, las fluctuaciones transitorias de los tiempos de coherencia (T_1 y T_2), y los errores en las compuertas entrelazadoras CNOT pueden desviar la convergencia del resolutor cuántico hacia mínimos locales subóptimos que violan las restricciones del espacio HPO como se ha visto al pasar el algoritmo VQE de simulación a Hardware cuántico real en la QPU del Qmio.

- **Especificidad de la Arquitectura Cuántica:** Los perfiles de resiliencia cuántica observados están ligados a procesadores superconductores de tipo *transmon* y su asimetría de ruido (donde las operaciones CNOT son más ruidosas que las rotaciones locales). Otras tecnologías alternativas (como trampas de iones o átomos neutros) con conectividades *all-to-all* y distintos perfiles de error alterarían la eficiencia relativa de VarQITE frente a otros resolutores.
- **Limitaciones del Modelado Cuadrático (QUBO):** El modelado HPO se formula sobre una aproximación cuadrática de segundo orden ajustada mediante regresión Lasso L1. Si el paisaje de rendimiento del `EventTransformer` presentase interacciones de tercer orden o superiores entre hiperparámetros (por ejemplo, correlaciones triples complejas entre `BATCH_SIZE`, `EMB_SIZE` y `ATTN_HEADS`), el modelo QUBO no podría capturarlas nativamente sin añadir qubits auxiliares y aumentar la profundidad del circuito. Lo cual limita el verdadero óptimo, pudiendo llegar a encontrar múltiples variables relacionadas, y teniendo que limitarse el estudio a simplemente ir descubriendo las combinaciones óptimas de dos en dos.

7.3. Vías de Mejora y Trabajo Futuro

La validación física de los resolutores cuánticos en la QPU superconductora de Qmio abre líneas de investigación futuras para expandir el potencial académico y operativo de este proyecto:

- **Implementación de Técnicas Activas de Mitigación de Errores:** Se plantea integrar técnicas de mitigación activa de errores en Qiskit, como la Extrapolación de Ruido Cero (ZNE) o la Cancelación Probabilística de Errores (PEC). Esto permitiría corregir analíticamente las mediciones del Hamiltoniano de Ising reduciendo el impacto del ruido despolarizante en Qmio, sin el coste en profundidad de circuito que requiere la corrección de errores clásica.
- **Exploración de Ansatz Dinámicos y Adaptativos (*ADAPT-VarQITE*):** Se propone sustituir la estructura estática actual del circuito por esquemas adaptativos del estado del arte como *ADAPT-VQE* o *ADAPT-VarQITE*. Bajo

este enfoque, el circuito crece de forma dinámica agregando compuertas entrelazadoras CNOT únicamente en las direcciones que optimizan el gradiente energético, minimizando la profundidad cuántica y la exposición a la decoherencia física de los cúbits.

- **Ampliación de la Frontera Multiobjetivo de Green AI:** Se plantea expandir la formulación multiobjetivo del Hamiltoniano (que actualmente equilibra la precisión de Levenshtein y el tiempo de entrenamiento en GPU). Incorporar directamente en el QUBO métricas como el consumo de memoria RAM de la GPU o la huella de carbono de los entrenamientos permitiría utilizar VarQITE como motor de codiseño de modelos de Deep Learning eficientes y sostenibles.
- **Evaluación Comparativa en Procesadores de Distinta Naturaleza Física:** Sería de gran interés validar el pipeline en otras tecnologías cuánticas de hardware. Comparar la convergencia del integrador de paso fijo de VarQITE en arquitecturas de trampas de iones (con conectividades *all-to-all*) o átomos neutros frente al procesador de cúbits transmon de Qmio permitiría determinar qué plataformas físicas son más eficientes para resolver problemas HPO a escala industrial.

Bibliografía

- [1] A. J, A. Adedoyin, J. Ambrosiano et al., «Quantum algorithm implementations for beginners,» *ACM Transactions on Quantum Computing*, vol. 3, n.º 4, págs. 1-92, 31 de dic. de 2022.
- [2] S. Rashid, «Understanding quantum computing and its policy implications,» mar. de 2026.
- [3] C. Hughes, J. Isaacson, A. Perry, R. F. Sun y J. Turner, «What is a qubit?» En *Quantum Computing for the Quantum Curious*. Cham: Springer International Publishing, 2021, págs. 7-16.
- [4] I. Y. Doskoch y M. A. Man'ko, «Superposition principle and born's rule in the probability representation of quantum states,» *Quantum Reports*, vol. 1, n.º 2, págs. 130-150, 26 de sep. de 2019.
- [5] A. A. Gorbatshevich y N. M. Shubin, «Quantum logic gates,»
- [6] M. Bataille, *Quantum circuits of CNOT gates*, 16 de dic. de 2020.
- [7] S. Nishio, Y. Pan, T. Satoh, H. Amano y R. V. Meter, «Extracting success from IBM's 20-qubit machines using error-aware compilation,» *ACM Journal on Emerging Technologies in Computing Systems*, vol. 16, n.º 3, págs. 1-25, 31 de jul. de 2020.
- [8] S. Zhang y L. Li, «A brief introduction to quantum algorithms,» *CCF Transactions on High Performance Computing*, vol. 4, n.º 1, págs. 53-62, mar. de 2022.
- [9] «Quantum-inspired Optimization Algorithms for Scalable Machine Learning Models,» en, *International Journal of Intelligent Engineering and Systems*, vol. 18, n.º 10, págs. 275-290, nov. de 2025.
- [10] J. Jarsania y C. Chavda, «Quantum-Inspired Algorithms for Large-Scale Data Analytics: A Tensor Network Approach,» en, en *2026 IEEE 16th Annual Computing and Communication Workshop and Conference (CCWC)*, Las Vegas, NV, USA: IEEE, ene. de 2026, págs. 0317-0323.
- [11] M. Shan, Y. Halimu, J. Sun, W. Fang y V. Palade, «A swarm intelligence optimization algorithm on riemannian manifolds,» en, *Swarm and Evolutionary Computation*, vol. 101, págs. 102319, feb. de 2026.

- [12] Y. Li, J. Guo, C. Zhang, X. Wang y L. Guo, «Hybrid intelligent model for strength prediction and parameter sensitivity analysis of cemented paste backfill,» en, *MetaResource*, págs. 1-21, 2026.
- [13] Q. Qiu, M. Wu, Q. Sun, X. Li y H. Xu, *A Novel Quantum Algorithm for Ant Colony Optimization*, en, arXiv:2403.00367 [quant-ph], mar. de 2024.
- [14] M. G. De Andoin y J. Echanobe, «Implementable hybrid quantum ant colony optimization algorithm,» en, *Quantum Machine Intelligence*, vol. 4, n.º 2, pág. 12, dic. de 2022.
- [15] P. A. Viana y F. M. d. P. Neto, *A Quantum Genetic Algorithm Framework for the MaxCut Problem*, en, arXiv:2501.01058 [quant-ph], ene. de 2025.
- [16] P. Sridhar y H. M. Kittur, «Analog Integrated Circuit Optimization With an Enhanced Adaptive Quantum Genetic Algorithm,» en, *IEEE Access*, vol. 14, págs. 19 205-19 223, 2026.
- [17] N. Xie, X. Lee, T. Chen, Y. Saito, N. Asai y D. Cail, «An Adaptive Weighted Qite-Vqe Algorithm for Combinatorial Optimization Problems,» en, en *2025 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Albuquerque, NM, USA: IEEE, ago. de 2025, págs. 208-218.
- [18] X. Jiang, Z. Sheng, C. Gong, W. Li y Z. Shuai, «Measurement-efficient ADAPT-VQE with the SOAP parameter optimizer,» *The Journal of Chemical Physics*, vol. 163, n.º 22, pág. 224 118, 14 de dic. de 2025.
- [19] J. Obst, F. Truger, J. Barzen y F. Leymann, «Parameter Transfers for Warm-Started QAOA:» en, en *Proceedings of the 1st International Conference on Quantum Software*, Bilbao, Spain: SCITEPRESS - Science y Technology Publications, 2025, págs. 37-48.
- [20] A. Lusso, C. N. Gimenez y A. Mata Ali, «Benchmarking QAOA on the Job Reassignment Problem: An Empirical Analysis using Transfer Learning and TQA Initialisation,» en, *SADIO Electronic Journal of Informatics and Operations Research*, vol. 25, n.º 1, e090, mar. de 2026.
- [21] E. Bae y S. Lee, «Recursive QAOA outperforms the original QAOA for the MAX-CUT problem on complete graphs,» en, *Quantum Information Processing*, vol. 23, n.º 3, pág. 78, feb. de 2024.
- [22] E. Jang, Z. Chen, D. Ha et al., «A Cyclic Layerwise QAOA Training,» en, *Quantum Machine Intelligence*, vol. 8, n.º 1, pág. 5, jun. de 2026, arXiv:2601.20029 [quant-ph].
- [23] G. Miki e Y. Tokura, *A New Scaling Function for QAOA Tensor Network Simulations*, en, arXiv:2505.23256 [quant-ph], mayo de 2025.
- [24] L. A. M. Rattighieri, P. M. Prado, M. C. d. Oliveira y F. F. Fanchini, *Measurement-Guided State Refinement for Shallow Feedback-Based Quantum Optimization Algorithm*, en, arXiv:2602.20407 [quant-ph], feb. de 2026.

- [25] M. Legnini y J. Berberich, «Robust Feedback-Based Quantum Optimization: Analysis of Coherent Control Errors,» en, en *2025 IEEE International Conference on Quantum Control, Computing and Learning (qCCL)*, Hong Kong, Hong Kong: IEEE, jun. de 2025, págs. 17-22.
- [26] J. V. Pankkonen, M. Raasakka, A. Marchesin e I. Tittonen, *Enhancing Hybrid Methods in Parameterized Quantum Circuit Optimization*, en, arXiv:2510.08142 [quant-ph], oct. de 2025.
- [27] L. Arceci, V. Kuzmin y R. V. Bijnen, *Gaussian process model kernels for noisy optimization in variational quantum algorithms*, en, arXiv:2412.13271 [quant-ph], dic. de 2024.
- [28] J. Pankkonen, L. Ylinen, M. Raasakka, A. Marchesin e I. Tittonen, *Gate Freezing Method for Gradient-Free Variational Quantum Algorithms in Circuit Optimization*, en, arXiv:2507.07742 [quant-ph], jul. de 2025.
- [29] Y. Sato, H. C. Watanabe, R. Raymond et al., «Variational quantum algorithm for generalized eigenvalue problems and its application to the finite-element method,» en, *Physical Review A*, vol. 108, n.º 2, pág. 022429, ago. de 2023.
- [30] I. Sajjad, H. M. Alshanbari, M. M. A. Almazah et al., «Adaptive Grover-driven optimization for quantum-inspired deep learning: A gradient-free training framework,» en, *AIMS Mathematics*, vol. 10, n.º 11, págs. 26568-26592, 2025.
- [31] H. Ominato, T. Ohyama y K. Yamaguchi, «Grover Adaptive Search With Fewer Queries,» en, *IEEE Access*, vol. 12, págs. 74619-74632, 2024.
- [32] N. H. Viet, N. X. Tung, T. Van Chien y W.-J. Hwang, «Advanced Quantum Annealing for the Bi-Objective Traveling Thief Problem: An ϵ -Constraint-Based Approach,» en, *IEEE Transactions on Quantum Engineering*, págs. 1-13, 2026.
- [33] N. Kowshik, S. Manna y S. P. Pal, *Quantum Approximate and Quantum Walk Optimization Approaches to Set Balancing*, en, arXiv:2509.07200 [quant-ph], sep. de 2025.
- [34] M. Zering, J. Joyce, T. Gurfinkel y J. Wang, *Benchmarking Lie-Algebraic Pretraining and Non-Variational QWOA for the MaxCut Problem*, en, arXiv:2512.22856 [quant-ph], dic. de 2025.
- [35] Y. Sun, J. Liu, Z. Wu, V. Tresp e Y. Ma, *SA-DQAS: Self-attention Enhanced Differentiable Quantum Architecture Search*, en, arXiv:2406.08882 [quant-ph], ago. de 2025.
- [36] S.-X. Zhang, C.-Y. Hsieh, S. Zhang y H. Yao, «Differentiable Quantum Architecture Search,» en, *Quantum Science and Technology*, vol. 7, n.º 4, pág. 045023, oct. de 2022, arXiv:2010.08561 [quant-ph].

- [37] Y. Sun, J. Liu, Y. Ma y V. Tresp, «Differentiable Quantum Architecture Search For Job Shop Scheduling Problem,» en, en *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Seoul, Korea, Republic of: IEEE, abr. de 2024, págs. 236-240.
- [38] Y. Li, R. Liu, X. Hao, R. Shang, P. Zhao y L. Jiao, «EQNAS: Evolutionary Quantum Neural Architecture Search for Image Classification,» en, *Neural Networks*, vol. 168, págs. 471-483, nov. de 2023.
- [39] S. B. Son y S. Park, «Q-RLONAS: Towards Efficient Quantum Neural Architecture Search,» en, en *2025 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Albuquerque, NM, USA: IEEE, ago. de 2025, págs. 1756-1765.
- [40] C. Chowdhury, «Exploring Quantum Support Vector Regression for Predicting Hydrogen Storage Capacity of Nanoporous Materials,» en, *Advanced Intelligent Discovery*, e202500015, nov. de 2025.
- [41] R. Ahmad, M. Kashif, N. Innan y M. Shafique, *Quantum vs. Classical Machine Learning: A Benchmark Study for Financial Prediction*, en, arXiv:2601.03802 [cs], ene. de 2026.
- [42] R. Coelho, A. Sequeira y L. Paulo Santos, «VQC-based reinforcement learning with data re-uploading: performance and trainability,» en, *Quantum Machine Intelligence*, vol. 6, n.º 2, pág. 53, dic. de 2024.
- [43] T. Yu y H. Zhu, *Hyper-parameter optimization: A review of algorithms and applications*, 12 de mar. de 2020.
- [44] J.-S. Hwang, S.-S. Lee, J.-W. Gil y C.-K. Lee, «Determination of optimal batch size of deep learning models with time series data,» *Sustainability*, vol. 16, n.º 14, pág. 5936, 12 de jul. de 2024.
- [45] P. Liashchynskiy y P. Liashchynskiy, «Grid search, random search, genetic algorithm: A big comparison for NAS,»
- [46] X. Wang, Y. Jin, S. Schmitt y M. Olhofer, «Recent advances in bayesian optimization,» *ACM Computing Surveys*, vol. 55, n.º 13, págs. 1-36, 31 de dic. de 2023.
- [47] F. F. Flöther, «The state of quantum computing applications in health and medicine,» *Research Directions: Quantum Technologies*, págs. 1-21, 24 de jul. de 2023.
- [48] A. Hassan, «Quantum-IoT security: Advances, challenges, and future directions,» *Journal of Hardware and Systems Security*, vol. 10, n.º 1, pág. 6, dic. de 2026.
- [49] J. Singh y K. S. Bhangu, «Contemporary quantum computing use cases: Taxonomy, review and challenges,» *Archives of Computational Methods in Engineering*, vol. 30, n.º 1, págs. 615-638, ene. de 2023.

- [50] A. A. Meléndez, C. G. Almudéver, M. A. Garcia-March et al., *Adaptive time compressed QITE (ACQ) and its geometrical interpretation*, Version Number: 1, 2025.
- [51] J. Buijs, «Environmental permit application process (‘WABO’), CoSeLoG project (version 1)[data set],» Technical report, Eindhoven University of Technology, inf. téc., 2014.

Apéndice A

Manuales técnicos

El código fuente íntegro desarrollado para este trabajo, así como los conjuntos de datos experimentales y los **scripts** de supercomputación, se encuentran disponibles de forma pública en el repositorio de GitHub: https://github.com/fedellor/TFG_QITE_HPO_Reproducibility.

Este repositorio ha sido diseñado bajo un paradigma de reproducibilidad estricta, conteniendo exclusivamente los archivos necesarios para ejecutar el **pipeline** algorítmico final y generar los resultados empíricos expuestos a lo largo de este documento.

A.1. Estructura de ficheros

A continuación, se detalla de forma exhaustiva la topología completa del repositorio, agrupada en carpetas funcionales:

- **Directorio Raíz:**
 - `README.md`: Documento de presentación, resumen del flujo metodológico y guía rápida de uso del repositorio.
 - `tfg.pdf`: Archivo de la memoria académica oficial en formato PDF.
 - `.gitignore`: Archivo de exclusión que filtra archivos temporales (como `__pycache__`), logs del clúster, entornos virtuales y metadatos de IDEs.
- **Módulo de Código Fuente (`src/`):** Agrupa la lógica ejecutable del pipeline y los procesadores:
 - `config.py`: Definición de hiperparámetros por defecto, nombres de marcas temporales, configuración de semillas pseudoaleatorias (7, 42, 99, 123, 1001) y rutas del clúster.

- `embeddings.py`: Implementación en PyTorch de la conversión de atributos del log de eventos a embeddings continuos unificados.
- `encoders_and_decoders.py`: Implementación de la arquitectura de red `EventTransformer` (bloques de autoatención multicabeza).
- `output_layers.py`: Capas lineales finales de proyección para predecir de forma multiobjetivo la siguiente actividad, tiempo de ciclo restante y recursos.
- `training.py`: Rutina de entrenamiento por lotes, optimización AdamW, decaimiento del gradiente y control de funciones de pérdida.
- `evaluation.py`: Algoritmo evaluador que calcula la exactitud median- te la distancia de similitud de Damerau-Levenshtein en secuencias pre- dictivas.
- `utils.py`: Funciones auxiliares de procesamiento de trazas, vectoriza- ción y padding.
- `sampler_hpo.py`: Script de muestreo uniforme del espacio continuo- discreto con filtro de divisibilidad de hardware.
- `train_hpo.py`: Script wrapper que gestiona el entrenamiento de 5 plie- gues y persiste atómicamente los resultados con bloqueo de archivos (`fcntl`).
- `generate_qubo.py`: Módulo de ajuste de regresión regularizada Lasso L1 LARS Cross-Validation para construir el QUBO.
- `solve_bayes_opt.py`: Solucionador clásico mediante procesos gaussia- nos.
- `solve_vqe.py`: Solucionador cuántico variacional VQE empleando el simulador MPS o la QPU real.
- `solve_qite.py`: Solucionador cuántico VarQITE resolviendo la ecua- ción de McLachlan con integrador de Euler de paso fijo.
- `generate_plots.py`: Módulo encargado de compilar los CSV y rende- rizar las gráficas del TFG.
- `train.ipynb`: Cuaderno Jupyter de soporte rápido para entrenamien- tos manuales e interactivos.
- `README.md`: Documento explicativo de la estructura del módulo de código fuente y la función de sus componentes.
- `data_processors/`: Carpeta que contiene la tubería de formateo y carga de logs:
 - `attributes.py`: Extractor de tipos de atributos categóricos y numéricos.

- `event_logs.py`: Cargador del log XES y mapeador de marcas de tiempo del proceso.
- **Gestión de Datos (data/)**: Almacenamiento de logs de eventos, matrices y pesos neuronales:
 - `resultados_hpo.csv`: Historial maestro con el pool clásico de HPO y sus 300 experimentos base.
 - `matriz_qubo.csv`: Matriz binaria simétrica Q (30x30) de acoplamiento cuántico.
 - `metricas_qubo.json`: Archivo que almacena la configuración del espacio, RMSE y el multiplicador de Lagrange λ .
 - `README.md`: Documento descriptivo de los conjuntos de datos, modelos entrenados y trazas del proceso.
 - `models/`: Carpeta que almacena los pesos preentrenados del predictor (`env_permit_fold0_event_predictor.pth`).
 - `event_log/`: Directorio con las trazas de minería de procesos:
 - `attributes.yaml`: Definición semántica de actividades, recursos y tiempos.
 - `env_permit.xes.gz`: Registro de eventos original comprimido.
 - `window_sizes.json`: Configuración de tamaño de ventana para el EventTransformer.
 - `split/`: Trazas divididas en 5 folds para validación cruzada, conteniendo los archivos en formato comprimido `train_foldX_env_permit.xes.gz`, `val_foldX_env_permit.xes.gz` y `test_foldX_env_permit.xes.gz` (donde $X \in \{0, 1, 2, 3, 4\}$).
- **Resultados e Imágenes (resultados/)**:
 - `resultados_soluciones_reales.csv`: Base de datos unificada que consolida las 55 soluciones.
 - `resultados_bayes_opt.csv`: Salida de la Optimización Bayesiana clásica (5 semillas).
 - `resultados_vqe_sweep.csv` y `resultados_qite_sweep.csv`: Resultados de barrido de ruido en FT3.
 - `resultados_vqe_qmio.csv` y `resultados_qite_qmio.csv`: Resultados físicos en la QPU de Qmio.
 - `README.md`: Documento explicativo de las tablas de resultados y los gráficos generados.
 - Gráficos de salida en alta resolución:

- `comparativa_tiempos.png`
 - `comparativa_energia.png`
 - `comparativa_factibilidad.png`
 - `impacto_ruido_factibilidad.png`
 - `promedio_violaciones.png`
 - `comparativa_tiempos_semillas.png`
 - `degradacion_tiempo_ruido.png`
 - `comparativa_energia_desglosada.png`
 - `frontera_pareto.png`
 - `comparativa_precision.png`
- **Orquestación HPC (slurm/):** Colección de scripts de envío a colas mediante Slurm:
 - `setup_env.sh`: Instalador automatizado del entorno virtual aislado con CUDA.
 - `launch_hpo.sh`: Script array de Slurm (tareas 1-100) para procesar el pool clásico HPO.
 - `run_bayes_opt.sh`, `run_vqe.sh`, `run_qite.sh`: Envíos para resolvers en la partición clásica.
 - `run_vqe_qmio.sh`, `run_qite_qmio.sh`: Enrutadores para ejecutar VQE y VarQITE en Qmio.
 - `README.md`: Documento explicativo de la configuración y orquestación de trabajos de supercomputación.
- **Documentación (docs/):**
 - `Formula_QUBO.png`: Render gráfico del Hamiltoniano de Ising.
 - `GrIA_TFG_Modelo_A_Memoria.pdf`: PDF con el borrador de la memoria.
 - `QUBO_MultiObjective_Formulation.md`: Borrador markdown de la formulación multiobjetivo.
 - `TFG_Master_Report_HPO.md`: Reporte maestro del proyecto.
 - `resumen_abstract.tex`: Archivo LaTeX del resumen de la tesis.
 - `manuales_tecnicos.tex`: LaTeX del presente Apéndice A.
 - `README.md`: Documento descriptivo de los informes, borradores y fuentes de documentación del proyecto.

A.2. Creación del entorno

Para asegurar la estabilidad en la reproducción de los cálculos y evitar conflictos de dependencias, se distingue el uso de dos entornos de ejecución en el clúster: por un lado, se utiliza **Python 3.10.8** para la fase clásica de aprendizaje profundo y entrenamiento HPO masivo sobre las GPUs NVIDIA A100 del FinisTerra III; por otro lado, se requiere estrictamente el uso de **Python 3.9.9** para la fase de resolución cuántica (tanto simulación como conexión directa a la QPU de Qmio) para garantizar la compatibilidad con el middleware de control cuántico `qmiotools`.

Las instrucciones para inicializar el entorno virtual desde una terminal en FT3 son:

1. Clonar el repositorio y acceder a la carpeta:

```
git clone https://github.com/fedellor/TFG_QITE_HPO_Reproducibility.git
cd TFG_QITE_HPO_Reproducibility
```

2. Ejecutar el script de aprovisionamiento de dependencias y activación en la partición Store de FT3:

```
chmod +x slurm/setup_env.sh
./slurm/setup_env.sh
```

A.3. Reproducción de los resultados

El pipeline de experimentos híbrido se reproduce siguiendo un orden secuencial estructurado:

1. **Generación del Pool HPO:** Ejecute el muestreo y envíe el array de trabajos a las GPUs A100 del FT3 para entrenar los 300 modelos Transformer base:

```
python src/sampler_hpo.py
sbatch slurm/launch_hpo.sh
```

2. **Ajuste del QUBO:** Una vez completados los entrenamientos clásicos, compile la superficie matemática cuadrática y obtenga la matriz QUBO:

```
python src/generate_qubo.py
```

3. **Ejecución de los Resolvedores (FT3 y Qmio):** Lance los resolvedores clásicos, simulaciones con ruido en FT3 o despliegues físicos en la QPU real de Qmio mediante los scripts de Slurm:

```
sbatch slurm/run_bayes_opt.sh  
sbatch slurm/run_vqe.sh  
sbatch slurm/run_qite.sh  
sbatch slurm/run_vqe_qmio.sh  
sbatch slurm/run_qite_qmio.sh
```

4. **Validación de Soluciones y Graficado:** Tras la finalización de los solucionadores, reentrene las soluciones óptimas inéditas en GPU y compile los diagramas de Pareto e histogramas ejecutando:

```
python src/generate_plots.py
```

Apéndice B

Licencia

Apache License
Version 2.0, January 2004
<http://www.apache.org/licenses/>

TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

"License" shall mean the terms and conditions for use, reproduction, and distribution as defined by Sections 1 through 9 of this document.

"Licensor" shall mean the copyright owner or entity authorized by the copyright owner that is granting the License.

"Legal Entity" shall mean the union of the acting entity and all other entities that control, are controlled by, or are under common control with that entity. For the purposes of this definition, "control" means (i) the power, direct or indirect, to cause the direction or management of such entity, whether by contract or otherwise, or (ii) ownership of fifty percent (50%) or more of the outstanding shares, or (iii) beneficial ownership of such entity.

"You" (or "Your") shall mean an individual or Legal Entity exercising permissions granted by this License.

"Source" form shall mean the preferred form for making modifications, including but not limited to software source code, documentation source, and configuration files.

"Object" form shall mean any form resulting from mechanical

transformation or translation of a Source form, including but not limited to compiled object code, generated documentation, and conversions to other media types.

"Work" shall mean the work of authorship, whether in Source or Object form, made available under the License, as indicated by a copyright notice that is included in or attached to the work (an example is provided in the Appendix below).

"Derivative Works" shall mean any work, whether in Source or Object form, that is based on (or derived from) the Work and for which the editorial revisions, annotations, elaborations, or other modifications represent, as a whole, an original work of authorship. For the purposes of this License, Derivative Works shall not include works that remain separable from, or merely link (or bind by name) to the interfaces of, the Work and Derivative Works thereof.

"Contribution" shall mean any work of authorship, including the original version of the Work and any modifications or additions to that Work or Derivative Works thereof, that is intentionally submitted to Licensor for inclusion in the Work by the copyright owner or by an individual or Legal Entity authorized to submit on behalf of the copyright owner. For the purposes of this definition, "submitted" means any form of electronic, verbal, or written communication sent to the Licensor or its representatives, including but not limited to communication on electronic mailing lists, source code control systems, and issue tracking systems that are managed by, or on behalf of, the Licensor for the purpose of discussing and improving the Work, but excluding communication that is conspicuously marked or otherwise designated in writing by the copyright owner as "Not a Contribution."

"Contributor" shall mean Licensor and any individual or Legal Entity on behalf of whom a Contribution has been received by Licensor and subsequently incorporated within the Work.

2. Grant of Copyright License. Subject to the terms and conditions of this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable copyright license to reproduce, prepare Derivative Works of, publicly display, publicly perform, sublicense, and distribute the Work and such Derivative Works in Source or Object form.
3. Grant of Patent License. Subject to the terms and conditions of

this License, each Contributor hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except as stated in this section) patent license to make, have made, use, offer to sell, sell, import, and otherwise transfer the Work, where such license applies only to those patent claims licensable by such Contributor that are necessarily infringed by their Contribution(s) alone or by combination of their Contribution(s) with the Work to which such Contribution(s) was submitted. If You institute patent litigation against any entity (including a cross-claim or counterclaim in a lawsuit) alleging that the Work or a Contribution incorporated within the Work constitutes direct or contributory patent infringement, then any patent licenses granted to You under this License for that Work shall terminate as of the date such litigation is filed.

4. Redistribution. You may reproduce and distribute copies of the Work or Derivative Works thereof in any medium, with or without modifications, and in Source or Object form, provided that You meet the following conditions:
 - (a) You must give any other recipients of the Work or Derivative Works a copy of this License; and
 - (b) You must cause any modified files to carry prominent notices stating that You changed the files; and
 - (c) You must retain, in the Source form of any Derivative Works that You distribute, all copyright, patent, trademark, and attribution notices from the Source form of the Work, excluding those notices that do not pertain to any part of the Derivative Works; and
 - (d) If the Work includes a "NOTICE" text file as part of its distribution, then any Derivative Works that You distribute must include a readable copy of the attribution notices contained within such NOTICE file, excluding those notices that do not pertain to any part of the Derivative Works, in at least one of the following places: within a NOTICE text file distributed as part of the Derivative Works; within the Source form or documentation, if provided along with the Derivative Works; or, within a display generated by the Derivative Works, if and wherever such third-party notices normally appear. The contents of the NOTICE file are for informational purposes only and

do not modify the License. You may add Your own attribution notices within Derivative Works that You distribute, alongside or as an addendum to the NOTICE text from the Work, provided that such additional attribution notices cannot be construed as modifying the License.

You may add Your own copyright statement to Your modifications and may provide additional or different license terms and conditions for use, reproduction, or distribution of Your modifications, or for any such Derivative Works as a whole, provided Your use, reproduction, and distribution of the Work otherwise complies with the conditions stated in this License.

5. Submission of Contributions. Unless You explicitly state otherwise, any Contribution intentionally submitted for inclusion in the Work by You to the Licensor shall be under the terms and conditions of this License, without any additional terms or conditions. Notwithstanding the above, nothing herein shall supersede or modify the terms of any separate license agreement you may have executed with Licensor regarding such Contributions.
6. Trademarks. This License does not grant permission to use the trade names, trademarks, service marks, or product names of the Licensor, except as required for reasonable and customary use in describing the origin of the Work and reproducing the content of the NOTICE file.
7. Disclaimer of Warranty. Unless required by applicable law or agreed to in writing, Licensor provides the Work (and each Contributor provides its Contributions) on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied, including, without limitation, any warranties or conditions of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A PARTICULAR PURPOSE. You are solely responsible for determining the appropriateness of using or redistributing the Work and assume any risks associated with Your exercise of permissions under this License.
8. Limitation of Liability. In no event and under no legal theory, whether in tort (including negligence), contract, or otherwise, unless required by applicable law (such as deliberate and grossly negligent acts) or agreed to in writing, shall any Contributor be liable to You for damages, including any direct, indirect, special, incidental, or consequential damages of any character arising as a result of this License or out of the use or inability to use the

Work (including but not limited to damages for loss of goodwill, work stoppage, computer failure or malfunction, or any and all other commercial damages or losses), even if such Contributor has been advised of the possibility of such damages.

9. Accepting Warranty or Additional Liability. While redistributing the Work or Derivative Works thereof, You may choose to offer, and charge a fee for, acceptance of support, warranty, indemnity, or other liability obligations and/or rights consistent with this License. However, in accepting such obligations, You may act only on Your own behalf and on Your sole responsibility, not on behalf of any other Contributor, and only if You agree to indemnify, defend, and hold each Contributor harmless for any liability incurred by, or claims asserted against, such Contributor by reason of your accepting any such warranty or additional liability.

END OF TERMS AND CONDITIONS

APPENDIX: How to apply the Apache License to your work.

To apply the Apache License to your work, attach the following boilerplate notice, with the fields enclosed by brackets "[]" replaced with your own identifying information. (Don't include the brackets!) The text should be enclosed in the appropriate comment syntax for the file format. We also recommend that a file or class name and description of purpose be included on the same "printed page" as the copyright notice for easier identification within third-party archives.

Copyright 2026 Diego Suárez Castro

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

<http://www.apache.org/licenses/LICENSE-2.0>

Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.